

DANIEL KEN KUMLIN

dk-capstone-v2

 Final Thesis

Document Details

Submission ID

trn:oid:::25824:375111550

Submission Date

Apr 8, 2026, 11:51 PM GMT+2

Download Date

Apr 9, 2026, 7:06 AM GMT+2

File Name

main.pdf

File Size

6.6 MB

58 Pages

15,319 Words

91,532 Characters

40% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



59 AI-generated only 40%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.





Ephemeral Interfaces: Design Patterns for Task-Scoped Generative UI Components

Daniel Kumlin

IE University

Capstone Project

Supervisor: Prof. *Jose Antonio Garcia Escandón*

Date (*April 2026*)

Acknowledgements

I would like to thank everyone who has helped guide me through the process of creating this thesis. It was a personal idea I really wanted to explore and felt empowered to try this with all the positive feedback I got from friends and colleagues.

I would like to thank my supervisor, Prof. Jose Antonio Garcia Escandón, for his guidance, feedback, and support throughout this capstone project.

I am also very grateful to the participants who conducted the study. It was an interesting twist to traditional surveys so I really appreciate everyone who took the time to take it.

Finally thank you for my family and friends who have supported me along the way.

Let the capstone begin.

Abstract

Generative UI has become a broader topic in software design because it promises interfaces that adapt to user goals instead of remaining static. Yet this promise collides with modern software use, where users still expect stable experiences, predictable controls, and durable mental models. This thesis investigates whether ephemeral, task-scoped generative UI components can offer a bounded alternative to full interface regeneration by appearing only when locally useful and disappearing without destabilising the host application.

The thesis develops and tests a framework for ephemeral support through a web-based research artifact. The main individual contributions are the framework operationalisation, a constrained component-specification model for temporary support, a validation pipeline for generated interface specifications, a bounded ephemeral overlay approach that preserves interface continuity, and an interactive survey artifact for empirical evaluation. The artifact was tested in a within-subjects user study comparing a baseline interface with an ephemeral condition on a bounded slide-reordering task.

The results show a mixed outcome. The ephemeral condition showed a descriptively higher task completion rate and significantly higher helpfulness ratings, while perceived control remained stable, but it did not improve completion time or overall preference and significantly increased interaction count and perceived intrusiveness. These findings suggest that ephemeral generative support is most credible not as a general replacement for existing interfaces, but as a bounded design strategy for bounded, low-consequence sub-tasks within broader workflows, especially where lightweight contextual guidance may be useful and interruption costs remain manageable. The thesis therefore contributes a framework, design guidelines, and an empirical basis for assessing when temporary generative assistance can add value without disrupting the user experience.

Keywords: generative AI, design, UI/UX, web apps, ephemeral interfaces, human-computer interaction, productivity software, design science research

Contents

1	Introduction	6
1.1	Problem and Motivation	6
1.2	Literature Review	7
1.3	Research Question and Contribution	9
1.4	Thesis Outline	9
2	Methodology	11
2.1	Research Design	11
2.2	Conceptual Framework	11
2.3	Artifact Development	14
2.4	Evaluation Design	16
2.4.1	Experimental Design	16
2.4.2	Participants	16
2.4.3	Materials and Procedure	16
2.4.4	Measures	17
2.4.5	Data Collection	18
2.4.6	Analysis Plan	18
3	Implementation	20
3.1	Architecture Overview	20
3.2	Participant Flow and Session Management	21
3.3	Data Model	22
3.4	Task Scenario Implementation	22
3.5	Ephemeral Support System	24
3.5.1	Component Specification Model	24
3.5.2	Generation and Validation Pipeline	26
3.5.3	Rendering Layer	29
3.6	Instrumentation and Logging	29

4	Results	31
4.1	Participants and Data Overview	31
4.2	Task Completion Rate	31
4.3	Task Completion Time	32
4.4	Interaction Count	32
4.5	Subjective Ratings	33
4.6	Overall Preference	34
4.7	Ephemeral Support Engagement	35
4.8	Summary of Findings	35
5	Discussion	37
5.1	Interpretation and Relation to Prior Work	37
5.2	Technical Iterative Refinement Contexts	39
5.3	Limitations	40
5.4	Robustness	42
6	Future Work	43
6.1	Broader scenarios and user populations	43
6.2	Deeper evaluation methods	44
6.3	Richer ephemeral component catalogues	44
7	Conclusion	45
A	Appendix	49
A.1	Analysis Pipeline Overview	49
A.2	Representative Analysis Listings	50
A.2.1	Participant Inclusion Logic	50
A.2.2	Paired Statistical Testing	50
A.2.3	Figure Generation	51
A.2.4	Thesis-Ready Value Generation	51
A.3	Local Generation Trace	51

A.4	Participant Flow Screenshots	54
A.5	Interpretive Note	57

1 Introduction

1.1 Problem and Motivation

Large language models are no longer confined to generating textual content; they are increasingly being used to produce user interfaces that adapt to the goals and context of individual users. This shift has opened a design space in which the structure of an application can respond to what a user needs rather than remaining fixed by its developers. Yet the promise of adaptive, AI-generated interfaces introduces a fundamental tension. On the one hand, static interfaces constrain users to predetermined layouts and user journeys that may not match the diversity of real-world tasks. On the other hand, fully generative approaches that replace the entire interface risk losing the benefits of a predictable and continuous interface that current day software provides.

This tension is especially acute in productivity software, where users build durable mental models of how their tools behave. When an interface is regenerated anew in response to each new request, orientation is lost so users are required to relearn the interface, how to use it, and where key controls are located. The design challenge, therefore, is not simply whether AI can generate useful interfaces, but how generative capability can be introduced into an existing application without eroding the stability that makes it usable in the first place.

A promising middle ground lies in *ephemerality*: rather than replacing an interface entirely, the system could surface temporary, task-scoped components that appear only when contextually justified and disappear once their purpose has been served. Such components would augment the host application at the point of need while leaving the surrounding interface intact. Although this idea draws on established principles from both generative UI research and interaction design theory, a coherent framework that specifies when ephemeral components should appear, how their scope should be bounded, and how they should be withdrawn without disrupting the user's workflow has not been articulated in prior design work.

1.2 Literature Review

While LLMs have been primarily used to generate content through text, they are becoming more prevalent in the generation of user interfaces (UI). It is a way to move beyond static interfaces by generating structures based on an underlying set of primitives tailored to the users' goals. Prior work suggests that when working with LLMs, users prefer to express themselves through UI instead of a plain chat. This is because the system can present its functionality properly instead of forcing it through text (Ahmed & Imran, 2025; Leviathan et al., 2025; Tang et al., 2025). However, most of the evidence comes from fully generated, end-to-end user interfaces, which differs from the design challenge addressed in this thesis, which is to augment existing applications where users expect a certain degree of stability and predictability in their tools. This raises the question of how an underlying set of core primitive UI components can be integrated into already established workflows instead of replacing the entire state of the interface with AI.

The problem with generative UI interfaces is that while they can increase the relevance of the task at hand, they can threaten predictability and user control. In real-world domains such as banking, their rigid interfaces were built on a set of predefined rules that fail to meet the ever-changing needs of their users across services (Tambi, 2020). Approaches that dynamically generate interfaces can address the problem of not tailoring just to the content needed to serve the user, but the actual structure of the interaction over time. Yet, these new systems raise an unresolved question: when generative UI is beneficial, how can it be introduced to users without it corroding the user's mental model of how the application should work? This means that the design guidelines for generative UI must balance adaptivity and consistency (Bieniek et al., 2024)

To manage this balance instead of having the entire interface generated by an LLM we restrict it to an ephemeral boundary. Thus, task-scoped components will only appear and disappear when needed. Past work on ephemeral interfaces argues that the permanence in ubiquitous environments can contribute to cognitive overload as there are too many controls and artifacts present (Döring et al., 2013). Conversely, ephemerality can reduce the persistent clutter of ar-

tifacts and help with better interactions for the user within a specific time frame and context. While this research was grounded in physical interaction design, it provides principles that can be transferred to digital systems. These ephemereal elements have to be easily discoverable, dismissable, and disappear without affecting the surrounding environment. Furthermore, it's a great use case for LLMs as the system could generate UI contextually in almost real time. (Bieniek et al., 2024). However, it must be kept in mind how the component lifecycle should be carefully designed to prevent an unstable user experience.

Ephemeral generative components have been implemented in past research without a full interface replacement, but with a temporary UI as an intermediate layer for exploration and understanding. For example, research was done in data analysis where they had to tweak their results from their machine learning models constantly through their code. To improve observability and comprehension of the LLM generated code an ephemereal interface was used instead of manually tweaking the values (Cheng et al., 2024). This shows how ephemereal UI can help users steer their outcome and iteratively refine stochastic tasks for exploring alternatives before taking action. However, this application of ephemeral UI was done in a narrow domain and intent. This leaves the question of how ephemeral patterns can generalise to everyday productivity applications where tasks are diverse. If it's not implemented properly, it could lead to more interruptions, context switching, or inconsistency of a familiar workflow.

Based on reviewed literature, it establishes three incomplete variables to the capstone. Firstly, a fully generative UI can improve user experience of interacting with applications compared to only chat-based interactions (Leviathan et al., 2025; Tang et al., 2025). Secondly, using dynamic interface generation in specific domains with ever evolving problems and rule based logic that require tailouring (Tambi, 2020). Thirdly, research on ephemerality shows that cognitive load can be reduced by decreasing the amount of permanent UI elements for the user. Nonetheless a gap remains at actually providing guidance on how to design these ephemereal, task-scoped generative UI components that can improve existing applications without affecting negatively the user experience. Prior work does not standardise reproducible design patterns of when generative components should appear, their bounded scope with an exit, and how they should preserve a predictable user interface. This capstone addresses those gaps by standard-

ising ephemerality into a framework of component patterns and rules to integrate into everyday applications.

1.3 Research Question and Contribution

The gap identified in the literature motivates the following research question:

How can ephemeral, task-scoped generative UI components be designed to improve existing applications without disrupting the user experience?

The capstone address this question in several contributions. Firstly it aggregates aa **conceptual framework** that defines a taxonomy of ephemeral components, a component lifecycle, and a set of design principles to constrain these temporary generative UI components in stable interfaces. Secondly, it **operationalises that framework** through a constrained component-specification model that defines what kinds of temporary support may be generated and how they may be structured. Third, it implements a **validation pipeline and bounded overlay approach** that checks generated specifications against structural and scenario constraints before rendering them as temporary, dismissible interface layers that preserve continuity with the host application. Fourth, it builds an **interactive survey artifact**: an experimental website that implements both a conventional baseline interface and an ephemeral condition governed by the framework's rules, with event logging and questionnaire capture built into the same system. Fifth, it evaluates the research through a **comparative within-subjects user study** that measure the measures task completion, efficiency, interaction effort, perceived helpfulness, intrusiveness, and user control across both conditions. These contributions go beyond existing literature by offering a empirical evidence of whether ephemeral UI can improve productivity scenarios

1.4 Thesis Outline

The remainder of this thesis is organised as follows. Section 2 presents the research methodology, including the conceptual framework, the design of the artifact, and the evaluation plan.

Section 3 describes the technical implementation of the experimental website and its support-generation pipeline. Section 4 reports the results of the user study across all behavioural and self-reported measures. Section 5 interprets the findings in relation to prior work and discusses the limitations and robustness of the evaluation. Section 6 outlines directions for future work, and Section 7 concludes.

2 Methodology

2.1 Research Design

Design Science Research (DSR) (Delport et al., 2024) was used as its primary goal was not only to show an existing phenomena but to create and evaluate an artifact capable of replicating the user identifying the design problem. The problem established in the literature was that current UI interfaces are either constrained completely to a static interface or the entire experience is dynamically generated resulting in unpredictably. Therefore, this research proposes the conceptual framework for ephemeral, task-scoped generative UI components to augment existing applications without disrupting the user experience. The methodological contribution lies in developing a framework that translates ideas from previous work on ephemerality and generative UI to guide better design decisions in a broader productivity context.

The study proceeds in three connected stages. Firstly, a framework based on existing literature on defining a taxonomy, lifecycle, and design principles for ephemeral interface components. Secondly, the framework is put into practice through a web-based research instrument artifact that allows these proposed ideas to be conducted in a controlled environment. Third, evaluation is conducted through the comparative user study by comparing a traditional interface with an ephemeral one on a single bounded task. By using the framework we can construct the artifact that will provide the basis for whether ephemeral UI can improve task performance and user experience.

2.2 Conceptual Framework

This study develops a conceptual framework for ephemeral, task-scoped generative UI components to guide both artifact design and evaluation. In one sentence, the framework specifies *which temporary support roles may appear, how those supports are triggered and withdrawn, and which constraints keep them useful without destabilising the host interface*. Positioning the framework here in the methodology is important because it defines the concepts that the artifact operationalises and the criteria against which the two interface conditions are later compared.

The first element is a taxonomy of component roles. Rather than classifying components by visual form, such as whether they appear as a panel, popover, or inline element, the taxonomy classifies them by the function they serve within the workflow. Three roles are distinguished. *Interpretive support* helps users understand context, available options, outputs, or likely consequences before acting. *Refinement support* helps users steer or configure an intended action before commitment, for example by comparing alternatives or adjusting parameters. *Task-execution support* helps users complete a bounded subtask inside the surrounding application without replacing the broader workflow. Framing these roles as a taxonomy is methodologically useful because it makes explicit what kind of assistance is being introduced in each scenario and avoids treating ephemeral behaviour as a single undifferentiated interface style.

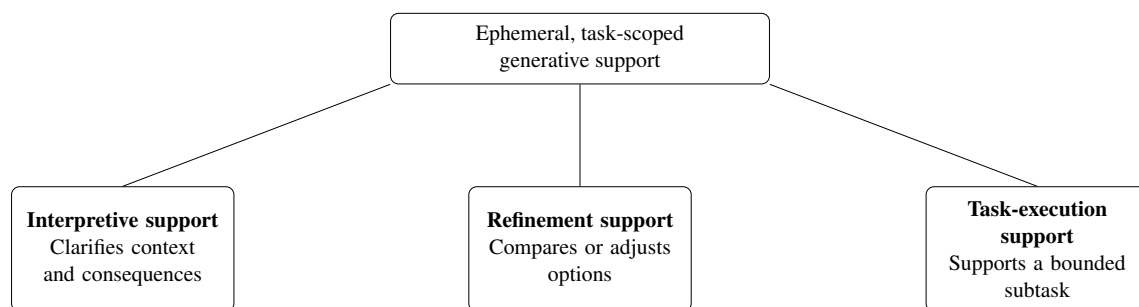


Figure 1: Taxonomy of ephemeral, task-scoped generative support roles.

The second element is a lifecycle describing how an ephemeral component enters and leaves use. Alternative stage patterns can be found in the literature, ranging from the minimal logic of temporary appearance and disappearance (Döring et al., 2013) to more scaffolded models in which users inspect and steer an intermediate layer before committing to an outcome (Cheng et al., 2024). For this study, the most appropriate synthesis is a four-stage lifecycle of *trigger*, *instantiation*, *interaction*, and *dismissal*. The component is triggered when a clear user goal or contextual need is detected that is not adequately supported by the persistent interface alone. That trigger may be explicit, such as a user request for help, or inferred from local behavioural and contextual signals, but only when the resulting support remains low-risk, relevant, and easy to dismiss. The component is then instantiated as a temporary interface element embedded within the existing application rather than replacing the surrounding interface. During interaction, users inspect, steer, or complete the immediate subtask while remaining oriented within the primary workflow. Finally, the component is dismissed once the task is

completed, abandoned, or no longer contextually relevant. This lifecycle was selected because it retains the conceptual clarity of ephemerality while remaining concrete enough to implement as a state-based interaction model in the artifact.

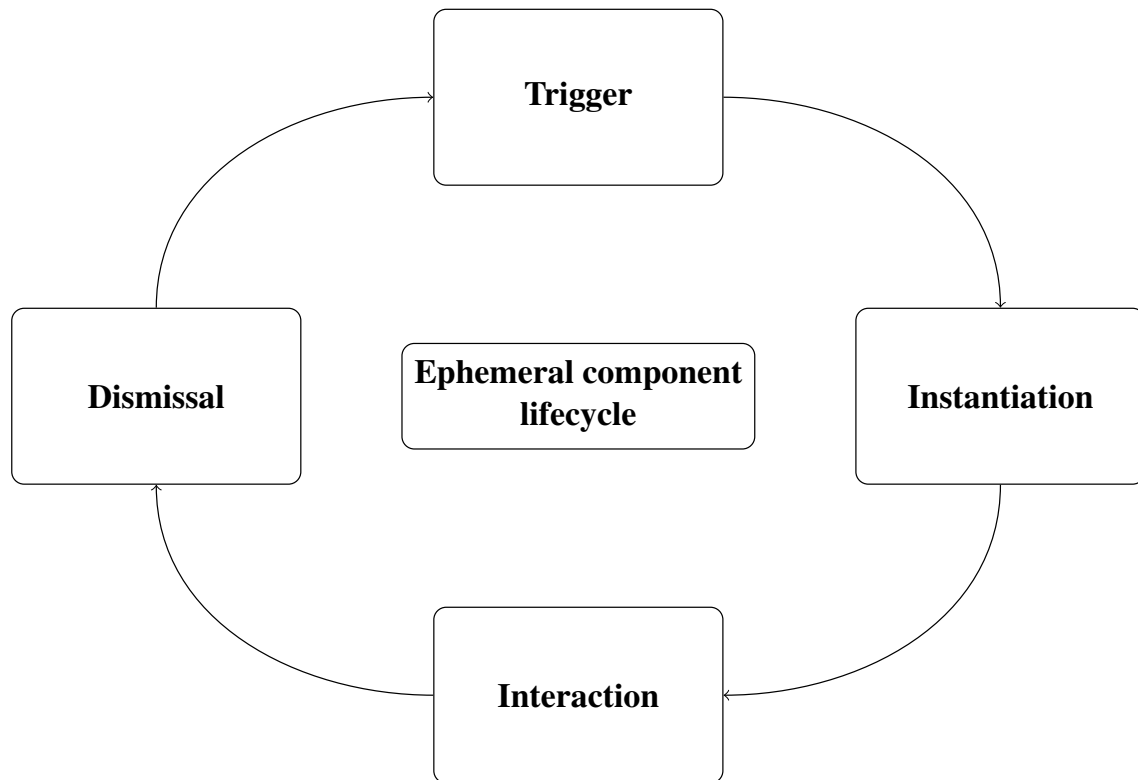


Figure 2: Lifecycle of an ephemeral, task-scoped generative UI component: trigger, instantiation, interaction, and dismissal.

The third element are design principles that constrain how ephemeral components should be introduced into a stable interface. Five principles guide the framework. **Contextual relevance** which determines the reason for when support should appear and serve a particular task. **Bounded scope** defines that the component should only remain present for the immediate task rather than the overall workflow. **Interface continuity** is where the component should preserve its surrounding elements so it feels part of the experience instead of redirecting the user to an unrelated experience. **User control** means that users should be able to inspect, refine, confirm, or dismiss the support rather than being forced to take a decision by the system. **Graceful disappearance** is used so that the component should disappear gracefully once it is not relevant to the immediate task at hand. If the ephemeral support is inferred rather than invoked explicitly then the defined principles constrain inferred assistance so consequential actions such as performing a destructive action to delete data is done without explicit user

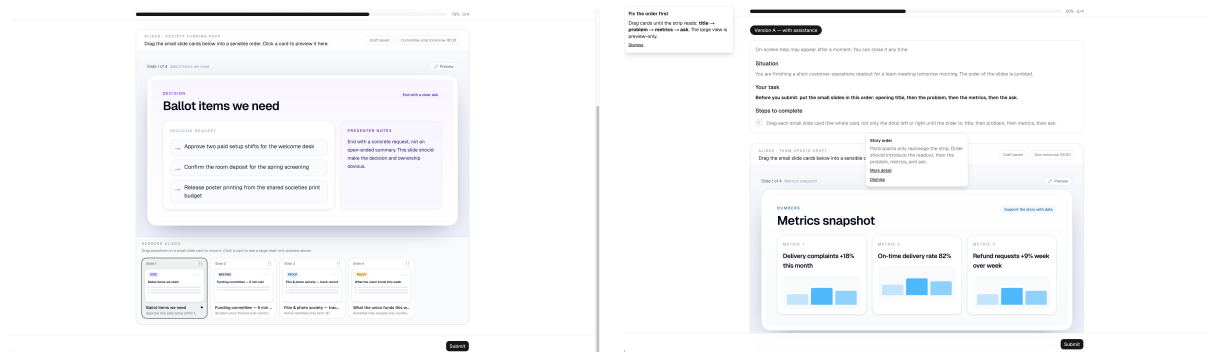
confirmation. Thus, these principles communicate the idea of ephemerality into practical design criteria that can be used consistently during the artifact development.

2.3 Artifact Development

The artifact developed in this project is a website used as a research instrument for evaluating the proposed framework in a controlled but realistic interaction setting. Rather than building a fully autonomous or fully generative application, the website is designed to preserve a recognisable baseline interface while allowing ephemeral, task-scoped components to be introduced at selected moments. This makes it possible to test whether the framework can augment the existing user journey without replacing the primary workflow altogether.

To operationalise the framework, the website will implement two interface conditions: an original condition using a well-known, conventional interface and an ephemeral condition in which temporary, task-scoped components are introduced according to the taxonomy, lifecycle, and design principles defined above. Both conditions use the same underlying slide-deck interface, task mechanics, and submission logic. The original condition presents the task without temporary support, whereas the ephemeral condition will surface temporary interface elements only when additional guidance, refinement, or task support is needed. In this way, the comparison isolates the effect of ephemeral augmentation against the non-ephemeral alternative.

The website presents one scenario that represents a bounded productivity task within the application: a short presentation-refinement exercise foregrounding *refinement support*. Participants receive a small slide deck whose order has been intentionally scrambled and must restore it to a coherent narrative sequence before submitting. The interaction is deliberately narrow: the main slide canvas functions as a large read-only preview, while the participant performs the task by reordering the slide thumbnails. The lifecycle of trigger, instantiation, interaction, and dismissal is implemented as the common interaction logic for both interface conditions. In the ephemeral condition, temporary support may propose order cues, comparisons, or lightweight explanations of narrative flow, but it does not edit slide content or take over the task. Success is therefore defined as submitting the deck in the predefined canonical order. Figure 3 shows the implemented study interface in both conditions.



(a) Baseline condition without temporary support. (b) Ephemeral condition with temporary support visible.

Figure 3: Screenshots of the implemented experiment interface. In both conditions, participants complete the same slide-reordering task: the large slide canvas is a read-only preview, while the actual task action is performed through the thumbnail strip. The ephemeral condition preserves the same underlying task structure but adds temporary, dismissible support overlays.

The ephemeral condition relies on a constrained support-generation pipeline rather than unrestricted interface generation. When support is requested or triggered, the system produces a structured interface specification from the current task context, validates it against schema and safety constraints, and either renders it as a temporary overlay or falls back to a deterministic alternative. The technical details of this pipeline, including the specification model, the validation stages, and the rendering layer, are described in Section 3.

Although the ephemeral condition is allowed to be visually and structurally expressive, that expressiveness remains bounded by the framework. Temporary support may add task-relevant interface elements, annotate or highlight existing content, expose comparison views, or locally reconfigure a small region of the page. However, it does not replace the full application, arbitrarily reorganise the entire interface, or perform consequential actions without confirmation. In practice, the role category, lifecycle, task goals, and success criteria are fixed in advance, while local details such as wording, defaults, and the exact form of low-risk support may vary in response to immediate context. The ephemeral version is therefore not intended to be wholly different for every participant. Instead, it is **constrained adaptation**: users encounter the same scenario structures and the same framework rules, but the temporary support may be parameterised slightly differently within those limits. This preserves analytical robustness while still allowing the artifact to reflect the broader motivation of adaptive, goal-relevant ephemeral interfaces.

2.4 Evaluation Design

2.4.1 Experimental Design

The framework will be evaluated through a comparative within-subjects user study using the website artifact described above. Each participant will complete two trials in total: the same slide-refinement scenario once as Version A and once as Version B. The presentation order of those versions is fixed, with Version A always shown first and Version B second, but the mapping between version label and experimental condition is randomised per participant so that one participant may receive the baseline first while another receives the ephemeral condition first under the alternative label. A within-subjects design is appropriate because it allows each participant to serve as their own control, thereby reducing the influence of individual differences in participants' digital literacy, task strategy, and previously acquired familiarity with similar interfaces. Having version labels preserves a consistent study flow while still varying which version corresponds to assistance across participants.

2.4.2 Participants

Participants will be recruited as regular users of web-based applications who can complete common digital tasks in a desktop browser environment. Recruitment will follow a convenience sampling approach appropriate to the scope of the capstone project. Inclusion criteria will focus on participants who are able to use standard web interfaces independently, while any exclusion criteria will be limited to factors that would prevent valid completion of the study tasks. The final sample size will depend on recruitment feasibility, but the study is designed to support repeated-measures comparisons between the two interface conditions.

2.4.3 Materials and Procedure

The main experimental artifact is the website. Participants complete the same scenario but with different context twice. The task is that they must rearrange a slide deck for a given correct sequence. The structure, goals, and success criteria remain constant for all users so that the study remains comparable across sessions. The ephemeral condition varies in how it's pre-

sented across sessions so that the assistance from the LLM can respond to given context while constrained to the predefined design rules from the framework. Thus, this study does not test unlimited personalisation or an unrestricted interface takeover. The main test is how bounded interfaces with temporary support could add, highlight, compare, or rearrange elements while preserving interface continuity and user control.

The procedure will be fully web-based. After opening the study website, participants will read the study information, provide consent, and complete a short background questionnaire about their comfort with websites and familiarity with AI-assisted tools. The website will then present two task trials in a fixed sequence of Version A followed by Version B, representing the same scenario under the two experimental conditions, with the condition-to-version mapping randomised per participant. Before each trial, participants will receive a short task prompt that labels the interface version. During each trial, the website will automatically log behavioural data. After both trials are done, participants will fill out a final comparative questionnaire about which condition they liked better, with a clear reminder of which version helped them. This fully instrumented procedure maintains the study's structure while facilitating the collection of all behavioural and self-reported data within a unified environment.

2.4.4 Measures

The evaluation combines both self-reported and behavioural measure to encapsulate task performance and perceived experience from the user. The Behavioural measures are derived from the interaction logs logged from the website. Subjective measures are collected through post-trial questionnaires. What is being tested is not only that ephemeral interfaces are preferred, but that they can actually improve how well a person can complete a task without external disruption. To assess that claim, efficiency measures task completion time, effectiveness through task completion rate, friction through interaction count. While user experience will be measured through ratings of intrusiveness, perceived control, helpfulness, and overall preference. Furthermore, ephemeral interactions like the time when ephemeral support is triggered, requested, shown, used, dismissed, ignored, or expanded for further detail is logged. It allows the study to gain further insight into how participants engaged with the generated assistance.

Table 1: Dependent measures and collection methods.

Measure	Type	Collection	Analysis
Task completion rate	Binary	Event logging	McNemar's exact test
Task completion time	Continuous	Timestamps	Wilcoxon signed-rank test
Interaction count	Discrete	Event logging	Wilcoxon signed-rank test
Intrusiveness	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Perceived control	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Helpfulness	Ordinal (1–7)	Post-trial Likert item	Wilcoxon signed-rank test
Preference	Nominal	Final forced choice	Binomial test

2.4.5 Data Collection

Data collection will combine instrumented event logging with questionnaire responses. For each trial, the website will record task start and end times, completion outcomes, interaction counts, submitted task state, and the active interface condition. In the ephemeral condition, logs from ephemeral interactions will capture support trigger events, explicit requests, support shown events, dismissals, usage confirmations, ignored support, detail expansions, and the generated support outputs themselves. Post-trial survey items will capture perceived intrusiveness, perceived control, and helpfulness, while the final questionnaire will capture overall preference between conditions. To link behavioural and survey data, each session is associated with a cookie-linked participant identifier that is stored consistently across the website logs and questionnaire responses. No names or email addresses are collected; the study stores only age range, occupation, familiarity ratings, trial logs, and questionnaire responses.

2.4.6 Analysis Plan

The primary analysis will compare the original and ephemeral conditions on each dependent measure using paired statistical tests selected for the data type, scale of measurement, and expected sample size.

Task completion rate is a paired binary outcome: each participant either completes the task correctly or does not, under each condition. This will be analysed using McNemar's exact test, which evaluates whether the proportion of participants who change completion status between conditions differs from what would be expected by chance. Completion proportions for each

condition will be reported alongside the test result.

Task completion time and interaction count are continuous and discrete measures, respectively, that are paired across conditions. Because the expected sample size is modest and time data is typically right-skewed, both measures will be analysed using the Wilcoxon signed-rank test, a non-parametric alternative to the paired t-test that does not assume normality of the paired differences. Medians and interquartile ranges will be reported for each condition.

Post-trial Likert ratings for intrusiveness, perceived control, and helpfulness are ordinal measures on a seven-point scale. Each rating will be analysed using the Wilcoxon signed-rank test, which is appropriate for paired ordinal data. Medians will be reported rather than means to respect the ordinal nature of the scale.

Overall preference is a forced-choice measure in which each participant selects which condition they preferred. This will be analysed using a binomial test against the null hypothesis that preference is split equally between conditions. The count and proportion choosing each condition will be reported.

Ephemeral-only interaction logs will be analysed descriptively to show the proportion of participants for whom support was triggered, shown, used, dismissed, ignored, or expanded, which helps interpret whether a null or positive effect reflects the quality of the support itself or low engagement. The analysis will therefore focus on whether the ephemeral condition yields improvements in efficiency, effectiveness, or perceived helpfulness without increasing intrusiveness or reducing perceived control. Because the ephemeral experience may vary within bounded framework rules, the analysis treats that variation as part of the intended adaptive behaviour of the artifact rather than as an uncontrolled change to the task itself.

3 Implementation

The previous section defined the conceptual framework, the artifact's purpose as a research instrument, and the evaluation design that structures the comparative study. This section describes how those ideas were translated into a working system. The goal is not to document every engineering decision, but to make the technical design legible enough that the reader can assess whether the artifact faithfully operationalises the framework and whether the instrumentation is sufficient to support the evaluation.

3.1 Architecture Overview

The artifact is built as a full-stack web application using the Next.js App Router with TypeScript (Vercel, 2026b). The server layer exposes a set of API routes that handle participant management, trial orchestration, event logging, questionnaire storage, and ephemeral support generation. Persistent data is stored in a PostgreSQL database. The ephemeral support system relies on Google Gemini, accessed through the Vercel AI SDK, to produce structured interface specifications from the current task state (Google, 2026; Vercel, 2026a).

These choices reflect the requirements of a research instrument rather than a production application. Next.js (Vercel, 2026b) was selected because its App Router collocates server-side logic and client rendering in the same project, which simplifies the instrumentation of both request handling and user interaction. TypeScript and Zod provide compile-time and runtime type safety across the generation pipeline, which is critical when validating AI-generated output against a constrained specification. PostgreSQL and Drizzle offer relational integrity and migration tooling appropriate for structured experimental data. The Vercel AI SDK (Vercel, 2026a) abstracts provider-specific details while supporting structured output modes, which allows the generation step to request JSON that conforms to a predefined schema rather than parsing free-form text.

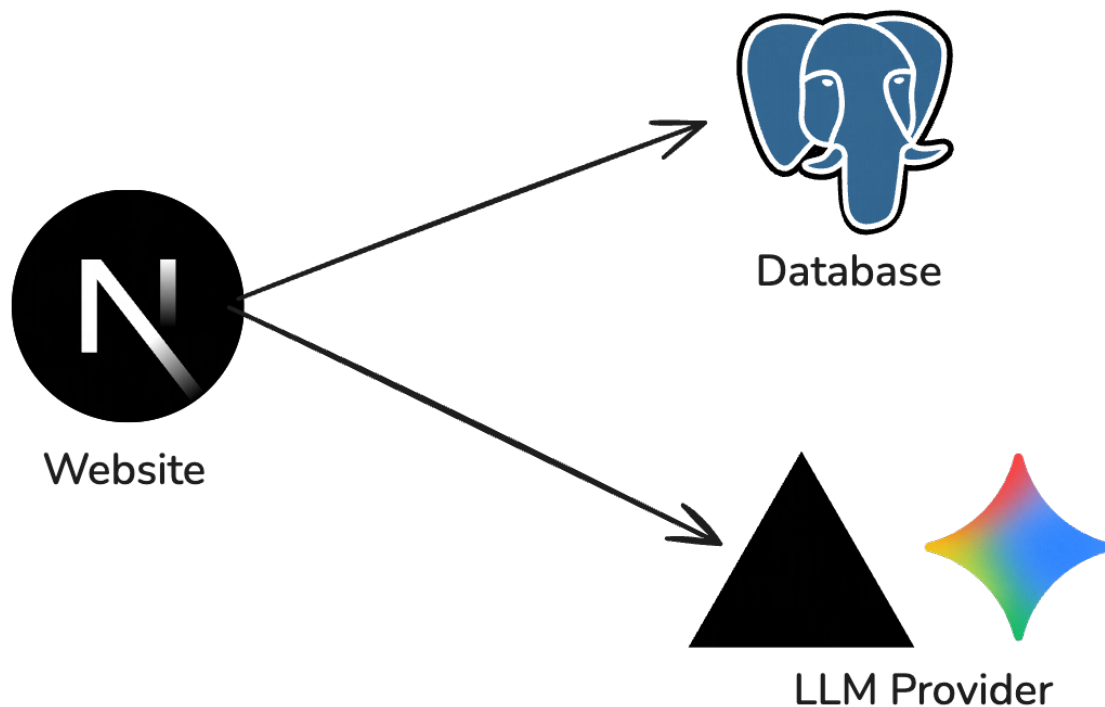


Figure 4: High-level system architecture of the experimental artifact.

3.2 Participant Flow and Session Management

The study procedure described in the methodology is implemented as a linear state machine that determines which page the participant should see at any point during the session. When a participant first opens the website, they are presented with the study information and consent form. Providing consent creates a participant record in the database and sets an HTTP-only session cookie that associates all subsequent requests with that record. No traditional authentication is required because the study collects no directly identifying personal data; the cookie serves only to link a browser session to its corresponding participant row.

After consenting, the participant completes a short background questionnaire covering age range, occupation, familiarity with web applications, and familiarity with AI-assisted tools. Submitting the background questionnaire acknowledges the study instructions and triggers the creation of the first trial row. From this point forward, a server-side state resolution function inspects the participant's record, their trial rows, and their questionnaire responses to determine the current study step. The possible steps form a fixed sequence: *background*, *study* (an active trial), *post-trial questionnaire*, *final questionnaire*, and *complete*. The client-side routing func-

tion maps each step to the corresponding page, so that refreshing the browser or navigating away always returns the participant to the correct point in the procedure.

Trial rows are created incrementally rather than all at once. The first trial is created when the participant acknowledges the instructions. Each subsequent trial is created only after the previous trial has been completed and its post-trial questionnaire has been submitted. This ensures that the participant cannot skip ahead or access later trials before finishing earlier ones.

Counterbalancing is implemented at the participant level. When a participant record is created, a boolean flag is randomly set to determine whether the baseline condition corresponds to Version A or Version B. The trial plan then assigns conditions to version labels using this flag, so that one participant may encounter the baseline first while another encounters the ephemeral condition first, both under the neutral label “Version A.” The content variant for each condition is assigned by hashing the participant and scenario identifiers, so the baseline and ephemeral trials use different slide decks.

3.3 Data Model

The database schema consists of five tables designed to capture the behavioural and self-reported data required by the evaluation measures defined in the methodology. Figure 5 shows the entity–relationship diagram (ERD): each participant may complete several trials; each trial may emit many logged events and many stored ephemeral support generations; questionnaire responses belong to a participant, and may optionally reference a trial (post-trial items) or omit a trial key (final questionnaire). Table 2 summarises what each table contributes to the evaluation.

The schema preserves referential integrity and supports the study queries needed for export and analysis.

3.4 Task Scenario Implementation

The study evaluates one scenario, the slide-reordering task described in the methodology. The implementation uses a scenario registry where each scenario’s metadata is defined, including its taxonomy category, correct answer, allowed ephemeral targets, task prompts, and a function

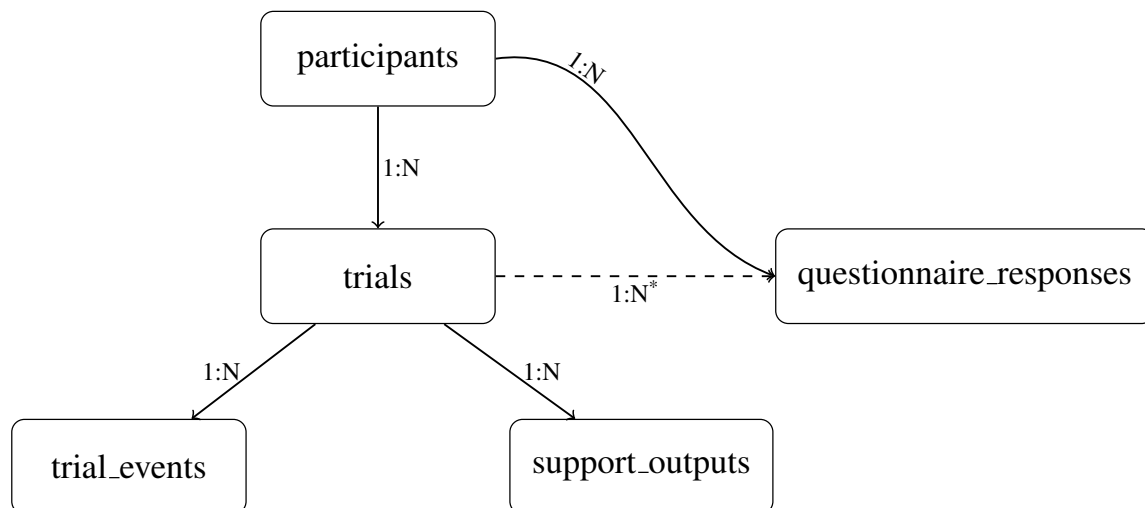


Figure 5: Entity–relationship diagram of the study database. Directed edges are labelled with cardinalities: one participant to many trials; one trial to many events and many support records; one participant to many questionnaire rows. The dashed edge indicates an optional foreign key: `trial_id` is NULL for final-questionnaire rows.

that builds the initial task state. This registry-based design means that adding a new scenario requires only a new registry entry rather than changes to the study flow or state machine.

The slide-reordering scenario presents participants with a short deck of four slides whose order has been intentionally scrambled. The main canvas area displays a read-only preview of the currently selected slide, while the actual task interaction occurs through a horizontal strip of draggable thumbnail cards. Drag-and-drop is implemented using the `@dnd-kit` library, which provides accessible keyboard and pointer-based reordering with sortable constraints. Participants reorder the strip and submit when they believe the sequence is correct.

Correctness is evaluated by comparing the submitted slide order against a predefined canonical order. Each scenario variant defines its own canonical sequence: variant A follows a corporate readout structure (title, problem, metrics, ask), while variant B follows a committee funding structure (meeting frame, policy, proof, ballot items). The variant assignment uses a deterministic hash of the participant identifier and scenario identifier to assign variant A or variant B to the baseline condition, with the opposite variant assigned to the ephemeral condition. This ensures that each participant encounters two different slide decks across the two trials, preventing familiarity effects from the task content while keeping the task structure identical.

Both conditions share the same task component, the same interaction mechanics, and the same submission logic. The only difference is whether the ephemeral support layer is active.

Table 2: Summary of each table’s role relative to the evaluation measures.

Table	Role in the study
participants	Consent and background questionnaire; counterbalancing flag (<code>baseline_is_version_a</code>); links all rows for a session.
trials	One row per task trial: condition, scenario, timing, correctness, submitted answer, interaction count—supports performance and efficiency measures.
trial_events	Typed events with JSON payloads—supports interaction counts and ephemeral engagement (e.g. support shown, dismissed).
questionnaire_responses	Post-trial Likert items and final preference; <code>trial_id</code> set for post-trial rows, null for final rows—supports subjective measures.
support_outputs	Ephemeral condition only: model input state and output spec per generation—supports analysis of support quality and fallback.

This design isolates the effect of ephemeral augmentation from other variables.

3.5 Ephemeral Support System

The ephemeral support system is the core technical contribution of the artifact. It translates the framework’s concepts of taxonomy, lifecycle, and design principles into a constrained generative pipeline that produces temporary interface overlays without replacing the host application. This subsection describes the three layers of the system: the specification model that defines what can be generated, the pipeline that produces and validates specifications, and the rendering layer that displays them.

3.5.1 Component Specification Model

Ephemeral support is represented as a typed tree structure called an `EphemeralSpec`. Each specification contains a version number, a root node, and metadata indicating whether the overlay is dismissible. Nodes are recursive: each node has a type, a property bag validated against a per-type schema, and optional children. The tree is constrained to a maximum depth of four levels and a maximum of six children per node. The decision to use JSON as the intermediate structure for these specifications was inspired by JSON Render, which similarly demonstrates how interface descriptions can be expressed declaratively and then rendered from structured data (Jason Lengstorf, 2026).

Table 3: Ephemeral component catalog. Each type is validated against a per-component Zod schema that enforces field types, string lengths, and structural constraints.

Category	Component type	Role
Visual cues	HighlightRing	Amber ring around a target element
	PulseRing	Animated ring with configurable duration
	ArrowCue	Arrow pointing at a target
	FocusMask	Dims everything except the target
Anchored annotations	AnchoredTooltip	Plain-text tooltip near a target
	HintStack	Ordered list of short hints near a target
	InspectPanel	Expandable panel with summary and optional detail lines
	ConsequenceNote	Single italic consequence line near a target
	SideNote	Margin annotation beside a target
Relational	ConnectorLine	Dashed line between two targets
	ComparisonStrip	Short contrast text positioned under two targets
	StepRail	Numbered callouts across two to six targets
Layout containers	Stack	Vertical sequence container
	ViewportPanel	Absolutely positioned panel in viewport percentages
	TargetOffsetPanel	Panel positioned relative to a target with pixel offsets
Rich content	FlowHtml	Sanitised HTML block (only inside ViewportPanel)
	AnchoredHtml	Sanitised HTML anchored near a target

The component catalog defines seventeen component types, organised into five functional categories. Table 3 lists the types and their roles. Visual cue components draw attention to specific interface elements without adding content. Anchored annotation components attach textual or rich content near a target element. Relational components connect or compare two or more targets to support reasoning across elements. Layout containers position children within the viewport or relative to a target. Rich content components render sanitised HTML and inline SVG for more expressive explanations such as flow diagrams or annotated illustrations.

Each component type is associated with a Zod schema that validates its properties at runtime. These schemas enforce constraints such as maximum string lengths, numeric ranges, and required fields. For example, an `InspectPanel` requires a target identifier, a title of at most 120 characters, a summary of at most 280 characters, and optionally up to four detail lines of at most 200 characters each. These limits prevent the model from generating excessively long or unbounded output while leaving enough room for contextually relevant content.

The catalog is versioned so that changes to the available component types or their con-

straints can be tracked across study sessions.

3.5.2 Generation and Validation Pipeline

When ephemeral support is requested during a trial, the system executes a multi-stage pipeline that generates a specification, validates it against the catalog and scenario constraints, and either accepts it or falls back to a deterministic alternative. Figure 6 illustrates this pipeline.

The generation step constructs two prompts. The system prompt describes the specification schema, lists all available component types with their property schemas, states the allowed target identifiers for the current scenario, provides taxonomy-specific guidance (such as which components are most appropriate for refinement support), and enumerates the design rules that constrain output structure. The user prompt includes the current task state, including slide order, deck metadata, and review goals, together with an optional participant interaction snapshot that reflects the participant's progress at the moment support was requested. This snapshot allows the model to tailor its output to what the participant has already done rather than generating generic first-visit guidance.

The system sends these prompts to Google Gemini using the Vercel AI SDK's structured output mode (Google, 2026; Vercel, 2026a), which constrains the model to produce JSON conforming to a provided Zod schema. The schema used for model output is a simplified subset of the full specification schema, designed to avoid compatibility issues with the provider's structured output implementation while remaining strict enough to prevent most structural errors at generation time.

The model's output then passes through a series of server-side validation stages:

1. **Envelope parsing.** The top-level structure is checked for a valid version number, root node, and metadata.
2. **Node tree parsing.** The root and its descendants are recursively validated for correct node shape, child count limits, and maximum depth.
3. **Per-component property validation.** Each node's properties are validated against its type-specific Zod schema, enforcing field types, string lengths, and numeric ranges.

4. **Target allowlisting.** Every target identifier referenced in the specification is checked against the scenario's predefined list of allowed targets. Any reference to an interface element not declared in the scenario registry causes the specification to be rejected.
5. **Structural container rules.** Layout containers are checked to ensure they contain at least one child. This applies to `Stack`, `ViewportPanel`, and `TargetOffsetPanel`.
6. **Ancestry constraints.** `FlowHtml` nodes are permitted only within a `ViewportPanel` subtree, preventing free-floating rich HTML from appearing anywhere in the interface.
7. **HTML sanitisation.** Any HTML content in `FlowHtml`, `AnchoredHtml`, or `SideNote` nodes is passed through a server-side sanitisation step using `DOMPurify` with a strict allowlist of tags and attributes. This prevents injection of scripts, event handlers, or disallowed elements while permitting semantic HTML and inline SVG for diagrams.

If any stage rejects the output, the system falls back to a deterministic, hardcoded specification for the current scenario. Fallback specifications are defined per scenario and per variant, ensuring that every participant in the ephemeral condition receives some form of support regardless of model behaviour. The fallback specifications use the same component types and target identifiers as model-generated output, so they appear visually consistent with the rest of the ephemeral experience.

Whether the specification was model-generated or a fallback, the full input state, model name, and output are persisted to the `support_outputs` table. This logging enables post-study analysis of generation quality, fallback frequency, and the relationship between input context and output content.

To verify the implementation end-to-end outside the aggregate study analysis, a local support-generation workflow was also conducted on 2026-04-04. Three consecutive hesitation-triggered requests for the slide-reordering scenario completed successfully and were persisted as support-output records without fallback. A non-trivial multi-component overlay was generated, validated, saved to PostgreSQL, and rendered successfully. Exact trace details are provided in Appendix A.3.

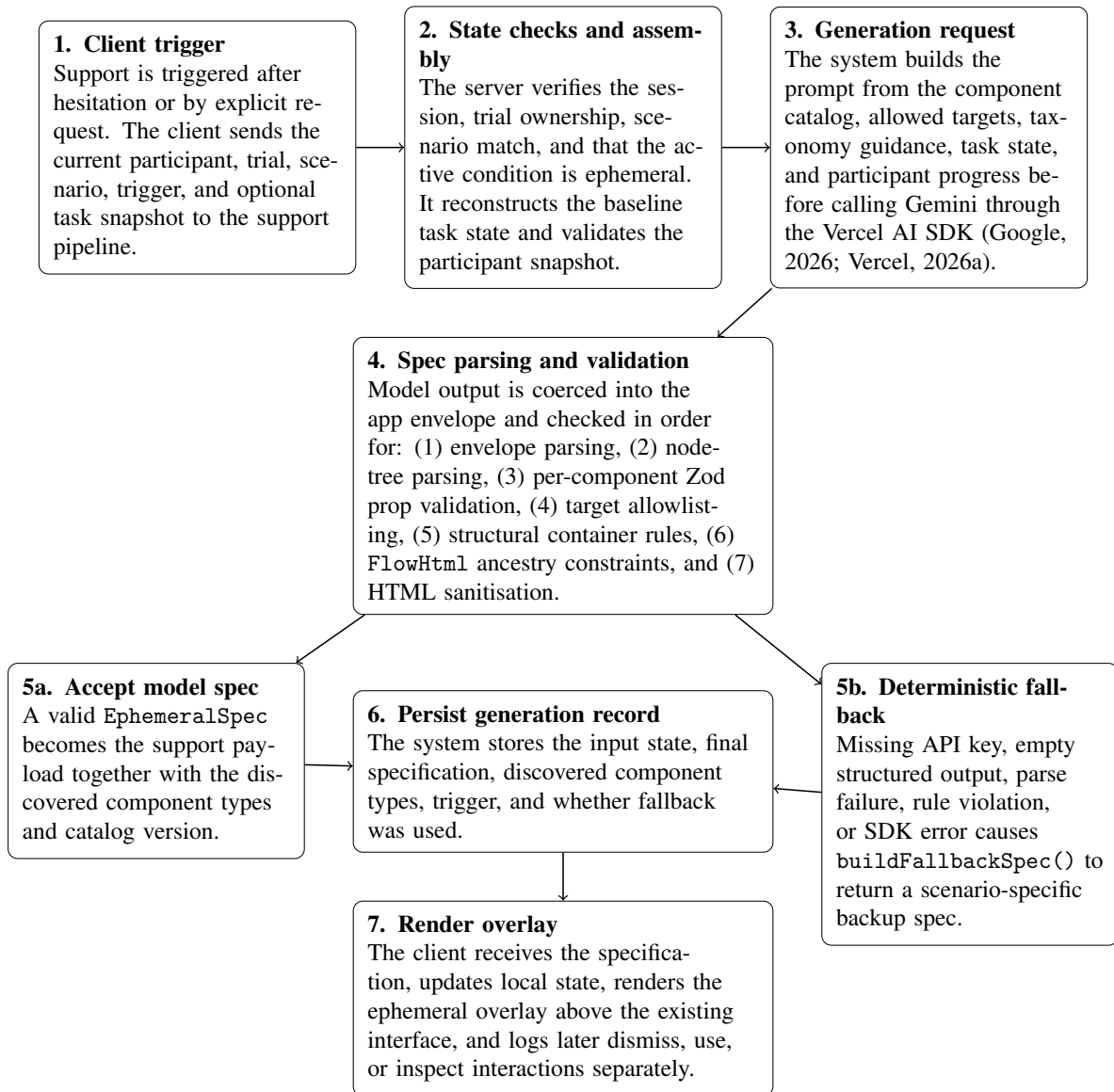


Figure 6: Ephemeral support generation and validation pipeline. Model output passes through seven validation stages before rendering; invalid output triggers a deterministic fallback. Both paths are logged for later analysis.

3.5.3 Rendering Layer

Accepted specifications are rendered as a temporary overlay above the existing application interface using a React portal (Meta Open Source, 2026). The overlay component receives the validated specification tree and recursively dispatches each node to its corresponding catalog component based on the node's type. A concrete generated example is shown in Appendix A.3. This dispatch model means that the renderer does not need to know about the content of any particular specification; it only needs to map types to components and pass through validated properties.

Components that reference target elements, such as `HighlightRing`, `AnchoredTooltip`, or `ConnectorLine`, resolve their targets by looking up DOM elements by identifier. A custom hook observes the target element's bounding rectangle and updates the overlay position when the layout changes, for example when the browser is resized or when other elements shift. A viewport clamping utility ensures that anchored components remain visible even when their target is near the edge of the screen.

The overlay respects the lifecycle stage of dismissal defined in the framework. When the specification's metadata marks the overlay as dismissible, a close control is rendered that allows the participant to remove the support at any time. Dismissal fires a logged event so that the analysis can distinguish between support that was actively used and support that was dismissed without engagement. The overlay does not persist across page navigations or trial boundaries, which enforces the ephemerality principle at the session level.

3.6 Instrumentation and Logging

The evaluation design in the methodology specifies several behavioural and self-reported measures. The artifact implements a corresponding instrumentation layer that captures the data needed for each measure.

Client-side interaction events are sent to a logging API endpoint that inserts rows into the `trial_events` table. Each event includes a type label and a JSON payload with contextual details. A predefined set of event types is recognised as countable interactions, and the trial's

running interaction count is incremented automatically when one of these events is recorded.

Table 4: Logged event types and their relationship to evaluation measures. Events marked with an asterisk (*) increment the trial's interaction count.

Event type	When recorded	Evaluation measure
trial_viewed*	Participant opens a trial	Interaction count
slide_reordered*	A slide is dragged to a new position	Interaction count
answer_selected*	An answer is first chosen	Interaction count
answer_changed*	An answer is revised	Interaction count
support_requested*	Participant explicitly requests support	Support engagement
support_triggered*	Support is triggered automatically	Support engagement
support_shown*	Support overlay becomes visible	Support engagement
support_dismissed*	Participant closes the overlay	Support engagement
support_used*	Participant acts on the support	Support engagement
support_ignored*	Support is present but not interacted with	Support engagement
support_inspect_expanded*	Participant expands an InspectPanel detail section	Support engagement

For the ephemeral condition, additional data is captured beyond interaction events. Each call to the support generation endpoint persists the full input state (task state and participant snapshot), the model name, and the complete output specification to the support_outputs table. This allows the analysis to examine not only whether support was shown and used, but also what the system produced and why, including whether a fallback was triggered.

Questionnaire responses are stored as individual question–response pairs rather than as monolithic form submissions. Post-trial ratings for intrusiveness, perceived control, and helpfulness are linked to the specific trial they follow, while final comparative questions are linked only to the participant. This structure supports direct joins between behavioural data and subjective ratings at the trial level.

A data export endpoint produces a structured bundle of all study data, protected by a server-side secret. The export includes participant records, trial summaries with performance metrics, event logs, questionnaire responses, and support generation outputs, providing the complete dataset needed for the analysis plan described in the methodology.

4 Results

This section reports the outcomes of the comparative user study. All measures follow the analysis plan described in the methodology: behavioural data was extracted from the experiment database, and self-reported ratings were collected through post-trial and post-study questionnaires administered within the same website. Unless stated otherwise, paired comparisons use the Wilcoxon signed-rank test, medians and interquartile ranges (IQR) are reported for continuous and ordinal measures, and statistical significance is assessed at $\alpha = .05$.

4.1 Participants and Data Overview

A total of 48 participants accessed the study website. Of these, 23 completed both task trials and the final questionnaire and are included in the analysis. The remaining participants were excluded because they did not finish both conditions or did not complete the post-study questions. The reported results therefore describe the completer sample rather than all users who initially opened the study website. Table 5 summarises the background characteristics of the included sample.

Table 5: Participant background characteristics ($n = 23$).

Characteristic	Summary
Age range	19 (6); 20 (1); 21 (3); 22 (9); 23 (3); 24 (1)
Web application familiarity (1–7)	Mdn = 6.0, IQR = 6.0–7.0
AI tool familiarity (1–7)	Mdn = 6.0, IQR = 5.0–7.0
Condition order: baseline first	15 (65%)
Condition order: ephemeral first	8 (35%)

4.2 Task Completion Rate

Task completion was defined as submitting the slide deck in the correct canonical order. In the baseline condition, 14 of 23 participants (60.9%) completed the task correctly. In the ephemeral condition, 17 of 23 (73.9%) completed correctly. McNemar's exact test comparing discordant pairs yielded $p = 0.250$.

The ephemeral condition therefore showed a descriptively higher completion rate than the baseline, but the difference was not statistically reliable in the present sample.

4.3 Task Completion Time

Completion time was measured from the moment the participant started the trial to the moment they submitted their answer. Participants who completed the task in the baseline condition had a median time of 35.7 seconds (IQR: 16.4–73.4), compared to 56.1 seconds (IQR: 22.3–85.1) in the ephemeral condition. A Wilcoxon signed-rank test did not yield a statistically significant difference ($W = 125$, $p = 0.709$, $r = 0.094$). Although the ephemeral median was slightly higher, the result does not indicate a reliable difference in efficiency between conditions.

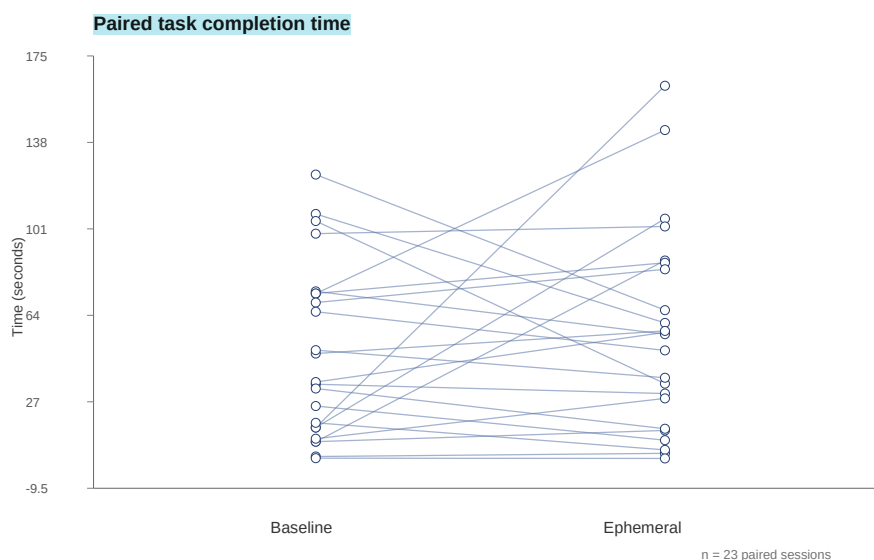


Figure 7: Paired comparison of task completion time (seconds) between the baseline and ephemeral conditions. Each line connects the same participant across conditions.

4.4 Interaction Count

The total number of logged interactions per trial was used as a proxy for task effort. The median interaction count in the baseline condition was 3.0 (IQR: 1.0–4.0), compared to 11.0 (IQR: 6.0–13.0) in the ephemeral condition. A Wilcoxon signed-rank test yielded a statistically significant difference ($W = 0$, $r = 1.000$; $p = <0.001$).

This higher count should be interpreted with caution because the ephemeral condition includes support-related events such as requesting support, support being shown, dismissal, and detail expansion. The increase therefore reflects greater overall interactivity, not only additional effort spent on the underlying slide-reordering task itself.

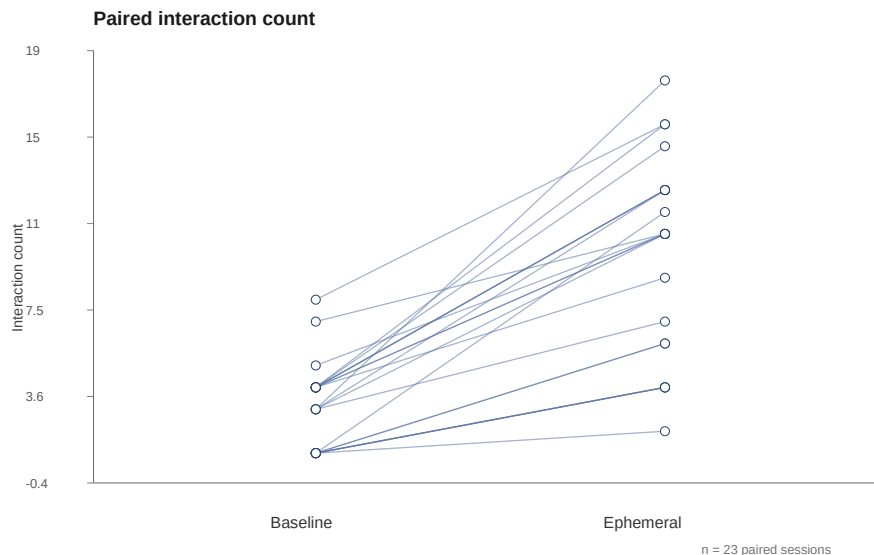


Figure 8: Paired comparison of interaction count between the baseline and ephemeral conditions.

4.5 Subjective Ratings

After each trial, participants rated the interface on three dimensions using a seven-point Likert scale. Table 6 summarises the results.

Table 6: Post-trial subjective ratings (7-point Likert scale, 1 = low, 7 = high).

Measure	Baseline		Ephemeral		Wilcoxon <i>p</i>
	Mdn	IQR	Mdn	IQR	
Helpfulness	5.0	3.0–6.0	6.0	5.0–6.5	0.029
Intrusiveness	3.0	1.5–5.0	5.0	4.0–6.0	0.004
Perceived control	6.0	4.5–7.0	6.0	4.5–6.0	0.579

The ephemeral condition received a higher median helpfulness rating than the baseline (6.0 vs. 5.0), and the Wilcoxon test indicated a statistically significant difference ($p = 0.029$). Thus, participants rated the ephemeral interface as more helpful.

Intrusiveness ratings were higher in the ephemeral condition (median 5.0) than in the baseline condition (median 3.0), and this difference was statistically significant ($p = 0.004$). Participants therefore experienced the temporary support as more noticeable or interruptive than the baseline interface.

Perceived control had the same median in both conditions (6.0 and 6.0), and the paired comparison was not significant ($p = 0.579$). Within the bounds of this study, the ephemeral support did not measurably reduce participants' sense of control over the task.

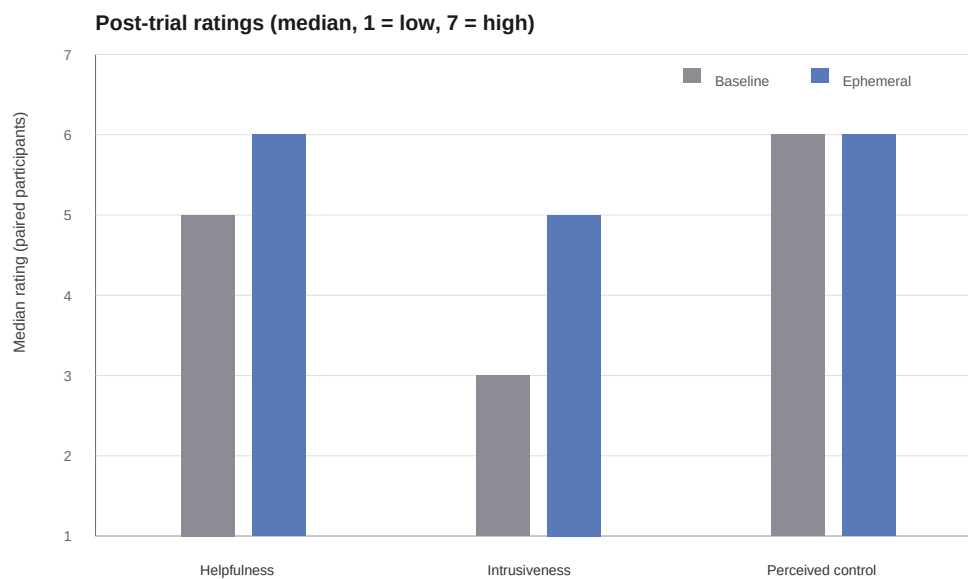


Figure 9: Median post-trial ratings by measure and condition (1 = low, 7 = high). Bars show the median rating across paired participants for each measure.

4.6 Overall Preference

After completing both trials, participants answered four forced-choice comparison questions. Responses were stored as Version A or Version B and mapped back to the actual experimental condition using each participant's counterbalancing assignment. Table 7 summarises the results.

Excluding the 6 participants who expressed no preference, 11 participants preferred the ephemeral condition and 6 preferred the baseline. A binomial test did not show a statistically

Table 7: Final comparative preference questions ($n = 23$).

Question	Baseline	Ephemeral	No preference
Overall preference	6	11	6
More helpful	8	12	3
More intrusive	1	18	4
Real-life preference	9	10	4

significant overall preference difference ($p = 0.332$), indicating that the sample did not meaningfully favour one condition over the other.

4.7 Ephemeral Support Engagement

To interpret the behavioural and subjective results in context, this subsection reports how participants actually interacted with the ephemeral support layer. These measures apply only to the ephemeral condition.

Of the 23 participants who completed the ephemeral trial, support was triggered or explicitly requested in 21 cases (91.3%). At the participant level, 6 participants recorded at least one `support_used` event, 21 recorded at least one `support_dismissed` event, 2 recorded at least one `support_ignored` event, and 3 recorded at least one detail-expansion event. These categories are not mutually exclusive: a participant could, for example, inspect or use support and later dismiss it in the same trial. The support generation pipeline produced 47 specifications in total, and no deterministic fallbacks were recorded (0 of 47, 0.0%).

This engagement pattern indicates that participants were usually exposed to the ephemeral support layer, but many chose not to engage deeply with it once it appeared. The condition-level outcomes should therefore be interpreted in light of partial uptake: the measured effects may reflect both the quality of the support itself and the fact that substantial portions of the offered assistance were dismissed after limited inspection or left unused altogether.

4.8 Summary of Findings

The ephemeral condition produced a higher descriptive task completion rate than the baseline, but the difference was not statistically significant. Completion time also did not differ signifi-

cantly, with the ephemeral condition showing a slightly higher median time. Interaction count was substantially higher in the ephemeral condition, indicating greater overall interface activity during the trial. Subjective ratings suggested that the ephemeral interface was significantly more helpful, significantly more intrusive, and no different in perceived control. Final preference responses did not reveal a statistically reliable overall preference for either condition. Engagement data shows that support was usually surfaced, but sustained use was limited, which provides important context for interpreting the mixed behavioural and subjective outcomes.

Table 8 provides an overview of all primary outcome measures.

Table 8: Summary of primary outcome measures across conditions.

Measure	Baseline	Ephemeral	<i>p</i>
Completion rate	60.9%	73.9%	0.250
Completion time (Mdn, s)	35.7	56.1	0.709
Interaction count (Mdn)	3.0	11.0	<0.001
Helpfulness (Mdn)	5.0	6.0	0.029
Intrusiveness (Mdn)	3.0	5.0	0.004
Perceived control (Mdn)	6.0	6.0	0.579
Overall preference	6 chose baseline; 11 chose ephemeral; 6 no preference		0.332

5 Discussion

This study evaluated whether ephemeral, task-scoped generative UI can improve a bounded productivity task. The final results show that it's a mixed answer. Relative to the baseline, the ephemeral version produced a higher descriptive completion rate, higher helpfulness rating, and stable control. However, completion time or overall performance did not improve. Furthermore, it significantly increased the interaction count and perceived intrusiveness. These findings suggest that ephemeral support can be integrated into modern day applications without disrupting the user experience and without neglecting user control. Though, this experiment as a whole was not able to deliver a clear performance gain without introducing more friction to the user.

5.1 Interpretation and Relation to Prior Work

This study's focus is different to most of the recent discussion around generative UI. Prior work emphasised on adaptability, multimodality, and the capability of these AI systems to tailor the interaction of users across contexts (Bieniek et al., 2024). Prior literature explains why static interfaces are insufficient in environments where user needs change rapidly. Additionally, it shows that the same adaptability that makes generative UI interesting can also threaten predictability, continuity, and user control. This study addresses that dilemma by going into a narrower alternative. Instead of replacing the entire interface dynamically, the artifact constrains the generative support to a temporary and task-scoped layer to avoid restructuring the entire user workflow. The results suggest this approach could provide benefits for the confidence of the user rather pure efficiency. Participants rated the ephemeral condition as significantly more helpful, but they did not complete the task faster and no reliable increase in success.

The results of this capstone support the claims that were deduced in the BISCUIT study (Cheng et al., 2024). It showed that ephemeral UIs can act as a intermediary between user intent and LLM-generated output in computational notebooks. In that study, the temporary interface made it easier for users to understand the code that was generated, look at other options, and lessen the work of prompt engineering. This work builds on that idea but moves it to a new

area. Instead of helping novice programmers who might rely completely on AI to help with the data analysis, it showed provides higher productivity gains for experienced programmers and preserve their orientation in an existing application. The weaker and more mixed effects seen here suggest that the current implementation was not as interactive as BISCUIT's notebook scaffolds. In this case, all participants encountered the support but only a fraction actually used it. Thus, lightweight overlays may not yield significant benefits unless they can provide immediate value to warrant that attention.

The discussion also connects to the earlier concept of ephemeral user interfaces proposed by Döring et al., who argue that intentionally temporary interface elements can reduce overload by limiting persistent accumulation and by presenting information only for the period in which it is relevant (Döring et al., 2013). Their work was grounded primarily in material and interaction-design perspectives rather than in web-based productivity systems, yet its core logic remains relevant here. The present thesis translates that logic into a digital software context by treating ephemerality not as a material property, but as a design constraint on interface behaviour. The subjective results highlight both sides of that design logic: perceived control remained stable, which supports the claim that the support stayed sufficiently bounded and did not take over the workflow, but intrusiveness increased significantly, showing that temporary appearance alone does not guarantee a low-disruption experience. In other words, bounded scope seems achievable, but trigger timing, presentation, and the immediate usefulness of the support remain critical if ephemerality is to feel genuinely unobtrusive.

All in all, these comparisons demonstrate that it's not needed to make a final verdict on the whether generative UI has a definitive place in productivity tools, but that it has a place in the middle in the literature. While static interfaces preserve continuity it struggles with flexibility and task specific assistance, fully generated interfaces provide unlimited flexibility with a risk of loss of user orientation and trust. The framework in this capstone proposes that ephemeral, task-scoped ui can offer a comprise between the two extremes. It does so by constraining the adaptive assistance through a boundaries of trigger conditions, scope, continuity, user control, and dismissal. The results suggest that the ephemeral support can be added without loss of perceived control. Nonetheless, it does indicate that the current intervention was too narrow,

intrusive, and weakly taken up to show stronger performance benefits.

In practical terms, the findings suggest a narrower conclusion than simply saying that ephemeral support suits productivity work in general. The safer interpretation is that it is most credible in bounded, low-consequence subtasks within broader workflows, especially where optional contextual guidance may help and the cost of interruption remains manageable. Examples include tasks that involve manually adjusting several variables and getting instant feedback, such as data analysis or design exploration. In these settings, temporary support can plausibly add value by helping users interpret local context, compare options, or choose the next step while still leaving the underlying application stable and familiar. At the same time, the current results suggest that such support must remain optional, lightweight, and immediately useful; otherwise the cost of interruption may outweigh the benefit of assistance.

The findings suggests that ephemeral support does not suit all productivity tasks in general, but in constrained specific subtasks. These tasks include manually adjusting variables for instant feedback like in data analysis or design exploration. Temporary support can add value while still leaving the underlying application stable and familiar. However, the current results suggest that this support should remain optional and immediately useful. Otherwise the cost of interruption can outweigh the benefit of assistance.

5.2 Technical Iterative Refinement Contexts

Although the thesis does not directly evaluate design tooling or data-analysis software, the pattern of results suggests that such contexts are worth treating as plausible next-step evaluation targets rather than as confirmed application domains. The benefit signal in the current study was not faster task execution, but higher perceived helpfulness under stable perceived control. That pattern is more consistent with interpretive or refinement-oriented value than with raw efficiency. For that reason, more technical workflows that involve iterative tweaking, immediate feedback, and bounded local decisions may be a stronger fit for future testing than workflows that depend on uninterrupted speed.

Examples include a designer adjusting layout, spacing, or hierarchy on a web page while previewing the result in real time, or a data analyst iteratively tuning parameters, filters, or

visual encodings while inspecting how the output changes. In both cases, the user remains inside a stable host interface and performs a sequence of locally scoped adjustments, which aligns well with the framework's emphasis on bounded scope, continuity, and user control. At the same time, this remains a hypothesis rather than a conclusion from the present data: the current artifact tested only a slide-refinement scenario, so the thesis can justify these technical tweaking contexts as promising next settings for evaluation, but not as the domain in which ephemeral support has already been shown to work best.

5.3 Limitations

An important boundary of the study is the scope of the evaluation itself. The artifact was evaluated through a single presentation-refinement scenario that foregrounded refinement support rather than the full range of support roles proposed in the framework. This choice was appropriate for the capstone because it kept the procedure manageable and reduced participant fatigue. A design with three scenarios across two interface conditions would have required six task trials per participant, which would likely have increased drop-off, reduced engagement, and weakened the quality of responses. However, this decision means that the findings cannot yet be generalised across multiple task types, domains, or support roles such as interpretive support or task-execution support.

The study is also limited by the breadth of interactions represented in the artifact. Some richer or more open-ended forms of support, such as more advanced writing assistance or more expressive temporary interaction patterns, were not retained in the final evaluation design. This means that the study primarily evaluates a bounded form of ephemeral support in a structured workflow rather than the broader design space of adaptive generative interfaces. As a result, the conclusions are strongest for short, goal-directed refinement tasks and should not be treated as evidence that the same interaction logic would perform equally well in longer writing workflows, highly creative tasks, or more complex multi-step activities.

Further limitations concern sampling, device conditions, and duration. The study relies on a modest participant sample recruited within the practical constraints of the project, which limits the diversity of backgrounds, habits, and expectations represented in the data. In addition, the

experimental artifact was not optimised for mobile use. If some participants accessed the study on smaller screens or in device conditions for which the interface was not well adapted, this may have introduced additional friction unrelated to the experimental condition itself and may therefore have skewed measures such as completion time, interaction effort, perceived intrusiveness, or overall preference. Participants also interacted with the system only within a short study session. This makes the evaluation suitable for examining immediate task performance and first impressions, but it does not capture longer-term effects such as habituation, changes in trust, or whether ephemeral assistance remains helpful once its novelty has worn off. The conclusions should therefore be read as evidence from a controlled short-term evaluation rather than as proof of long-term adoption in real-world settings.

Attrition also narrows the strength of the claims. Although 48 participants accessed the study website, only 23 completed both trials and the final questionnaire. This means the analysed sample likely overrepresents participants who were willing to continue through the full procedure, and the results should therefore be interpreted as evidence about completers rather than about everyone who initially encountered the study.

Another limitation is that the framework is tested through one particular implementation of the artifact rather than through multiple independent implementations. Positive or negative outcomes may therefore reflect not only the underlying framework, but also specific design decisions in this prototype, such as how support was phrased, when it was triggered, and how interaction options were presented. Subjective measures such as helpfulness, perceived control, and intrusiveness also rely on participant self-report and are therefore shaped by individual interpretation. These issues do not invalidate the findings, but they do mean that the results should be understood as an evaluation of a concrete operationalisation of the framework rather than a definitive test of ephemeral support in the abstract.

The robustness of the paired comparison is also bounded by the sample size and by the structure of the engagement data. Condition order was randomised across participants, which helps reduce systematic ordering bias, but the final sample is too small to support strong claims about subtle order effects. In addition, these support-engagement metrics capture overlapping event types rather than a single exclusive participant state, so they are most informative as

indicators of how often support was surfaced, interacted with, or dismissed, rather than as a complete behavioural taxonomy.

5.4 Robustness

At the same time, several aspects of the research design strengthen the robustness of the study within these boundaries. Both interface conditions were evaluated on the same underlying task with the same success criteria, which supports a fair comparison between the baseline and ephemeral versions. The within-subjects design reduced the influence of individual differences because each participant encountered both conditions, while randomising trial order helped limit systematic ordering effects. The website also combined automated event logging with post-trial questionnaires, making it possible to compare behavioural outcomes and subjective experience rather than relying on anecdotes alone. Finally, although the ephemeral condition allowed bounded adaptation, that variation was constrained by predefined framework rules concerning scope, lifecycle, and user control, which preserved analytical comparability across sessions.

One unexpected aspect of the findings is that the behavioural and subjective measures do not all point in the same direction. The ephemeral condition was rated as significantly more helpful and achieved a slightly higher completion rate, yet it also produced no speed advantage and was experienced as more intrusive. This combination suggests that participants may have recognised the intent or potential value of the support, with it still not feeling seamless enough to outweigh its interaction cost during a short task. That contradiction is important because it implies that future refinements may need to focus less on adding support in principle and more on making that support feel timely, lightweight, and worth engaging with. Taken together, these limitations and robustness measures suggest that the study should be interpreted as a focused but credible first evaluation of ephemeral, task-scoped support in a stable-interface software setting. The results are sufficient to discuss the promise of the framework in a controlled setting, but they do not establish broad generalisability across contexts. Section 6 sets out concrete directions for extending scenarios, evaluation methods, and the design space of ephemeral components.

6 Future Work

The present evaluation is intentionally narrow: one scenario, one operationalisation of the framework, and a remote, self-paced procedure. The directions below follow from that scope and from what the artifact already suggests about where ephemeral, task-scoped support may be most informative next.

6.1 Broader scenarios and user populations

The study used a single presentation-refinement workflow. Ephemeral support is unlikely to matter equally in every domain. Prior work such as BISCUIT suggests that temporary interface scaffolds can work well in computational notebook settings associated with programming and data-science exploration, where users benefit from inspecting and steering intermediate output (Cheng et al., 2024). BISCUIT is therefore a useful contrast for the present thesis: it evaluates ephemeral support in a notebook-centred exploratory workflow, whereas this project examines whether similar temporary support can add value inside a more conventional stable interface. The current results suggest that the next evaluations should focus less on broad domain labels and more on workflow properties: interruption tolerance, consequence of error, and whether the assisted action is a bounded local refinement rather than a high-stakes commitment. This makes technical iterative-refinement contexts especially interesting next cases to test, such as design exploration in which a user tweaks layout or hierarchy while previewing a web page, or data-analysis work in which a user adjusts parameters, filters, or visual encodings and immediately inspects the effect. This possibility also aligns with the broader generative-UI claim that adaptive support may be most valuable when conventional interfaces struggle to accommodate varying user needs across contexts (Bieniek et al., 2024; Tambi, 2020). A useful next step is therefore to evaluate the framework across several workflows that differ in interruption tolerance, consequence of error, task significance, technical complexity, and familiarity with the host application. Such work would help clarify when ephemeral support is genuinely complementary and when it merely duplicates guidance that users or existing tools already provide.

6.2 Deeper evaluation methods

Increasing the number of remote completions would improve statistical power, but it would not on its own address every open question about *why* participants behaved as they did, or how they interpreted ephemeral support moment to moment. A complementary direction is structured, in-depth evaluation: for example focus groups or moderated think-aloud sessions, where a researcher can observe navigation problems, misunderstanding of triggers, and emotional reactions that web logs and post-task questionnaires only partially capture. Combining a controlled quantitative comparison with a smaller set of rich qualitative sessions would better connect design choices to observed friction and to subjective measures such as perceived control and intrusiveness.

6.3 Richer ephemeral component catalogues

The evaluation artifact deliberately used a constrained set of ephemeral UI components so that conditions remained comparable and the study remained feasible within the capstone timeline. The underlying framework could support a wider catalogue: more varied layouts, stronger personalisation of content and tone based on the user's profile, role, or prior behaviour, and more expressive temporary interaction patterns than were deployed here. Exploring that wider design space would also connect more directly to the broader literature, which frames generative interfaces in terms of adaptive, context-sensitive interaction structures while also warning that flexibility can come at the cost of predictability and user control (Bieniek et al., 2024). At the same time, the foundational logic of ephemerality suggests that temporary interface elements should appear only when relevant and should recede without leaving persistent clutter (Döring et al., 2013). Expanding the catalogue therefore raises its own research questions about how to preserve predictability and user agency when surface behaviour varies more widely, and how to evaluate more personalised variants without confounding condition effects with implementation novelty. Future work could treat catalogue design as a first-class object of study: progressive disclosure of complexity, user-tunable boundaries for ephemerality, and systematic comparison of a small number of high-diversity implementations against a stable baseline.

7 Conclusion

This thesis asked how ephemeral, task-scoped generative UI components can be designed to improve existing applications without disrupting the user experience. To address that question, it developed a conceptual framework for ephemeral support, implemented the framework in an experimental web artifact, and evaluated the artifact against a persistent baseline in a within-subjects user study. The evaluation provides a qualified answer to that question: ephemeral support can be introduced in a bounded way that preserves users' sense of control, but the implementation tested here did not produce clear performance gains without also introducing additional intrusiveness.

Across the study measures, the ephemeral condition showed a slightly higher task completion rate, although that difference was not statistically significant, and a significantly higher perceived helpfulness rating. Completion time likewise did not improve. By contrast, interaction count increased substantially and perceived intrusiveness increased significantly, while perceived control remained stable and overall preference did not clearly favour either condition. These findings suggest that the main contribution of the ephemeral layer in this prototype was not faster task execution, but a clearer sense of perceived support that came with a noticeable interaction cost.

The thesis's individual contribution lies not only in evaluating ephemeral support as an idea, but in operationalising it through a constrained component-specification model, a validation pipeline for generated interface specifications, a bounded ephemeral overlay approach that preserves host-interface continuity, and an interactive survey artifact that makes empirical evaluation possible.

The broader implication is that ephemeral, task-scoped generative UI remains a promising design direction, but its value depends on more than simply making support temporary. For such support to improve existing applications in practice, it must appear at the right moment, be immediately legible, and offer enough value that users choose to engage with it rather than dismiss it. The present findings support a narrower conclusion than claiming a best-fit domain such as productivity work in general or technical design work specifically. The safer

conclusion is that ephemeral support is most credible for bounded, low-consequence subtasks within stable interfaces, particularly where lightweight contextual guidance may help and the cost of interruption remains manageable. The framework proposed in this thesis helps define those boundaries, while the study shows both their promise and their current limits. Technical iterative-refinement settings, such as design exploration or parameter tuning with immediate feedback, are plausible next contexts for evaluation, but they remain future directions rather than validated conclusions from the present study. Future work should therefore test richer scenarios, broader user populations, and more refined support behaviours in order to determine when ephemeral generative assistance becomes not only possible within stable interfaces, but meaningfully beneficial.

References

- Ahmed, A., & Imran, A. S. (2025). The role of large language models in ui/ux design: A systematic literature review. *arXiv preprint arXiv:2507.04469*. <https://arxiv.org/abs/2507.04469>
- Bieniek, J., Rahouti, M., & Verma, D. C. (2024). Generative ai in multimodal user interfaces: Trends, challenges, and cross-platform adaptability. *arXiv preprint arXiv:2411.10234*. <https://arxiv.org/pdf/2411.10234>
- Cheng, R., Barik, T., Leung, A., Hohman, F., & Nichols, J. (2024). Biscuit: Scaffolding llm-generated code with ephemeral uis in computational notebooks. *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. <https://arxiv.org/pdf/2404.07387>
- Delport, P. M. J., Von Solms, R., & Gerber, M. (2024). Methodological guidelines for design science research. *Procedia Computer Science*, 237, 195–203. <https://doi.org/10.1016/j.procs.2024.05.096>
- Döring, T., Sylvester, A., & Schmidt, A. (2013). Ephemeral user interfaces: Valuing the aesthetics of interface components that do not last. *ACM Interactions*, 20(4), 32–37. <https://interactions.acm.org/archive/view/july-august-2013/ephemeral-user-interfaces>
- Google. (2026). *Gemini api documentation*. Retrieved April 5, 2026, from <https://ai.google.dev/gemini-api/docs>
- Jason Lengstorf. (2026). *Json render*. Retrieved April 5, 2026, from <https://json-render.dev/>
- Leviathan, Y., Valevski, D., Kalman, M., Lumen, D., Segalis, E., Molad, E., Pasternak, S., Natchu, V., Nygaard, V., Venkatachar, S., Manyika, J., & Matias, Y. (2025). Generative ui: Llms are effective ui generators. *arXiv preprint arXiv:2410.11354*. <https://generativeui.github.io/static/pdfs/paper.pdf>
- Meta Open Source. (2026). *Createportal*. Retrieved April 5, 2026, from <https://react.dev/reference/react-dom/createPortal>
- Tambi, V. K. (2020). Generative ai applications in customizing user experiences in banking apps. *The Research Journal (TRJ)*, 6(6), 1–15.

- Tang, F., Xu, H., Zhang, H., Chen, S., Wu, X., Shen, Y., Zhang, W., Hou, G., Tan, Z., Yan, Y., Song, K., Shao, J., Lu, W., Xiao, J., & Zhuang, Y. (2025). A survey on (m)llm-based gui agents. *arXiv preprint arXiv:2504.13865*. <https://arxiv.org/abs/2504.13865>
- Vercel. (2026a). *Ai sdk*. Retrieved April 5, 2026, from <https://sdk.vercel.ai/>
- Vercel. (2026b). *Next.js*. Retrieved April 5, 2026, from <https://nextjs.org/>

A Appendix

This appendix documents the key analysis steps used to transform the study database into the statistics, figures, and thesis-ready result values reported in Section 4. Because the examiner cannot assume access to the full repository, the appendix includes selected code excerpts that demonstrate the core logic of participant selection, statistical testing, figure generation, and LaTeX value generation. The excerpts are representative rather than exhaustive: they are intended to show how the analysis was implemented, not to reproduce every utility function in full.

A.1 Analysis Pipeline Overview

The analysis workflow was implemented as a small Bun-based pipeline. Rather than copying values manually into the thesis, the pipeline exported study data from PostgreSQL, computed the required descriptive and inferential statistics, generated the figure PDFs used in the results chapter, and wrote the LaTeX macro file that supplied numeric values to the manuscript. This structure reduced transcription risk and kept the reported values tied to executable analysis code.

The pipeline can be summarised in four stages:

1. export the required study data to intermediate CSV files,
2. compute the paired statistical tests used in the results chapter,
3. generate the PDF figures included in the results chapter, and
4. regenerate the LaTeX macros that populate reported values throughout Section 4.

This means that the values appearing in the results chapter were derived from the exported data rather than entered by hand. If the underlying database contents changed, rerunning the pipeline updated both the figures and the numeric placeholders used in the LaTeX source.

A.2 Representative Analysis Listings

The following listings were selected because they show the most thesis-relevant parts of the implementation: the rule used to determine the analysed sample, the paired non-parametric statistical test used for several outcomes, the generation of the main numeric figures, and the production of thesis-ready LaTeX commands.

A.2.1 Participant Inclusion Logic

Listing 1 shows the reusable SQL filter used throughout the analysis. A participant was included only if they completed both task trials and answered all four final comparison questions. This ensured that all reported measures were computed from a consistent completer sample.

```
WITH valid_participants AS (  
  SELECT p.id, p.age_range, p.occupation,  
         p.web_app_familiarity, p.ai_tool_familiarity,  
         p.baseline_is_version_a  
  FROM participants p  
  WHERE (SELECT count(*) FROM trials t  
         WHERE t.participant_id = p.id AND t.completed = true) = 2  
        AND (SELECT count(DISTINCT question_key)  
             FROM questionnaire_responses qr  
             WHERE qr.participant_id = p.id  
                 AND qr.trial_id IS NULL  
                 AND qr.question_key IN (  
                   'final_preference',  
                   'final_helpfulness',  
                   'final_intrusiveness',  
                   'final_real_life'  
                 )) = 4  
)  
SELECT count(*) AS valid_completed  
FROM valid_participants;
```

Listing 1: SQL logic used to define the valid participant sample.

A.2.2 Paired Statistical Testing

Listing 2 shows the central implementation of the Wilcoxon signed-rank test used for paired completion time, interaction count, and post-trial Likert outcomes. The function removes zero

differences, ranks the remaining paired differences by magnitude, computes the smaller rank sum W , and then uses an exact p-value for small samples or a normal approximation for larger ones. This matched the within-subjects design and the small paired sample available in the study.

A.2.3 Figure Generation

Listing 3 shows the core drawing loop used to produce the paired numeric plots for completion time and interaction count. Each participant contributes one baseline value and one ephemeral value, and the script draws a line connecting the two observations so that the within-participant change is visible directly in the figure.

A.2.4 Thesis-Ready Value Generation

Listing 4 shows how the final numerical outputs were converted into LaTeX commands. Instead of entering p-values, medians, and percentages manually into the thesis text, the script generated commands such as `\GenTimeP` and `\GenCompPctBaseline`. The results chapter then imported these commands directly, ensuring that the prose, tables, and summary values all used the same generated source.

A.3 Local Generation Trace

In addition to the aggregate study dataset analysed in Section 4, I captured a local end-to-end validation trace from the implemented support pipeline itself. This trace was taken from the real `/api/support` route rather than the development-only playground endpoint, so it exercised the same request handling, prompt construction, validation stages, PostgreSQL persistence, and client-facing response path used during the study. The selected example below is the locally persisted record with `support_output_id = 103`, captured on 2026-04-04 during a hesitation-triggered `slides-outline-refine` trial.

```
function wilcoxonSignedRank(
  baseline: number[],
  ephemeral: number[],
): { W: number; p: number; r: number; n: number } {
  const diffs: number[] = [];
  for (let i = 0; i < baseline.length; i++) {
    const d = ephemeral[i]! - baseline[i]!;
    if (d !== 0) diffs.push(d);
  }

  const n = diffs.length;
  if (n === 0) return { W: 0, p: 1, r: 0, n: 0 };

  const absDiffs = diffs.map((d) => ({ abs: Math.abs(d), sign: Math.sign(d)
    → }));
  absDiffs.sort((a, b) => a.abs - b.abs);

  const ranks: number[] = [];
  let i = 0;
  while (i < absDiffs.length) {
    let j = i;
    while (j < absDiffs.length && absDiffs[j]!.abs === absDiffs[i]!.abs) j++;
    const avgRank = (i + 1 + j) / 2;
    for (let k = i; k < j; k++) ranks.push(avgRank);
    i = j;
  }

  let wPlus = 0;
  let wMinus = 0;
  for (let k = 0; k < n; k++) {
    if (absDiffs[k]!.sign > 0) wPlus += ranks[k]!;
    else wMinus += ranks[k]!;
  }

  const W = Math.min(wPlus, wMinus);
  const maxW = (n * (n + 1)) / 2;
  const p = n <= 25 ? wilcoxonExactP(W, n) : wilcoxonApproxP(W, n);
  const r = 1 - (2 * W) / maxW;

  return { W, p: Math.min(p, 1), r: Math.abs(r), n };
}
```

Listing 2: Core Wilcoxon signed-rank implementation used for paired outcomes.

```

const baselineVals = rows.map((r) => Number(r[opts.baselineKey]));
const ephemeralVals = rows.map((r) => Number(r[opts.ephemeralKey]));

for (let i = 0; i < rows.length; i++) {
  const b = baselineVals[i]!;
  const e = ephemeralVals[i]!;
  if (Number.isNaN(b) || Number.isNaN(e)) continue;
  const y1 = yToPdf(b, minY, maxY, chartBottom, chartTop);
  const y2 = yToPdf(e, minY, maxY, chartBottom, chartTop);
  page.drawLine({
    start: { x: xLeft, y: y1 },
    end: { x: xRight, y: y2 },
    thickness: 0.8,
    color: lineColor,
    opacity: 0.55,
  });
}

for (let i = 0; i < rows.length; i++) {
  const b = baselineVals[i]!;
  const e = ephemeralVals[i]!;
  if (Number.isNaN(b) || Number.isNaN(e)) continue;
  const y1 = yToPdf(b, minY, maxY, chartBottom, chartTop);
  const y2 = yToPdf(e, minY, maxY, chartBottom, chartTop);
  page.drawCircle({ x: xLeft, y: y1, size: 2.8, borderColor: pointColor });
  page.drawCircle({ x: xRight, y: y2, size: 2.8, borderColor: pointColor });
}

```

Listing 3: Core paired-plot drawing logic used for numeric result figures.

```
const lines: string[] = [
  "% AUTO-GENERATED by analysis/generate-results-tex.ts -- do not edit by",
  "  ↳ hand.",
  "% Regenerate: bun run gen-tex",
  "",
];

cmd(lines, "GenNTotal", String(s.overview.totalAccessed));
cmd(lines, "GenNValid", String(v));
cmd(lines, "GenCompPctBaseline", s.completion.pctBaseline.toFixed(1));
cmd(lines, "GenCompPctEphemeral", s.completion.pctEphemeral.toFixed(1));
cmd(lines, "GenCompMcNemarP", fmtP(s.completion.mcnemarP));

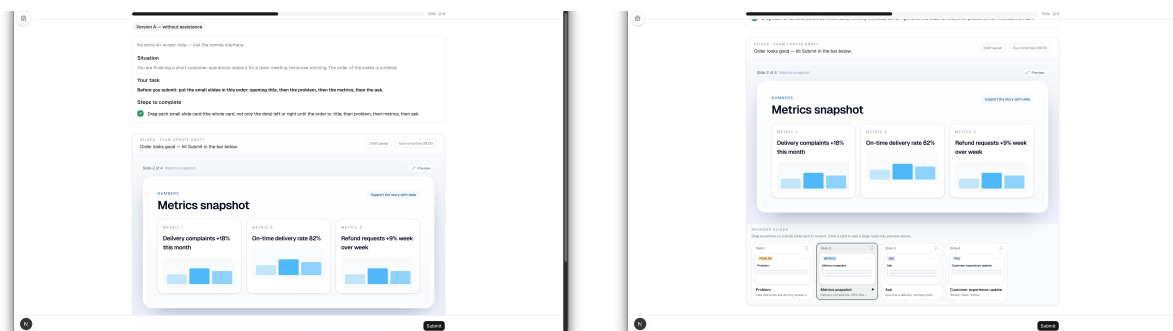
const tw = s.time.wilcoxon;
cmd(lines, "GenTimeBaselineMdn", safeFixed(s.time.baselineMdn, 1));
cmd(lines, "GenTimeEphemeralMdn", safeFixed(s.time.ephemeralMdn, 1));
cmd(lines, "GenTimeP", fmtP(tw?.p ?? null));

writeFileSync(OUT_TEX, lines.join("\n") + "\n", "utf-8");
```

Listing 4: Generation of LaTeX commands used throughout the results chapter.

A.4 Participant Flow Screenshots

Figures 10–14 provide a chronological walkthrough of the participant-facing flow captured from the implemented artifact. These appendix figures complement the main-text interface screenshots by showing the sequence of states the participant moved through across the baseline trial, the ephemeral trial, and the final comparative questionnaire. One intervening transition capture that contained no visible interface content was omitted.



(a) Baseline trial with the plain interface visible (b) Baseline task interaction while the participant reorders the slide strip.

Figure 10: Participant-flow walkthrough, part 1: baseline-condition task screens.

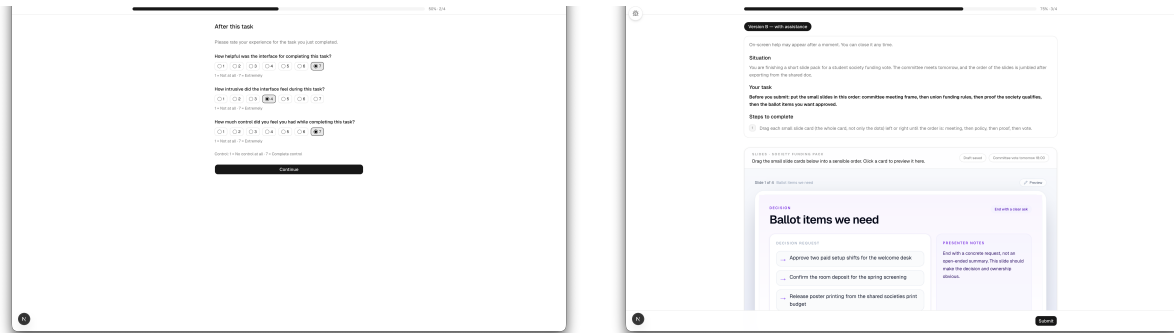
Ephemeral Interfaces: Task-Scoped Generative UI

55

```
{
  "date": "2026-04-04",
  "route": "/api/support",
  "scenarioId": "slides-outline-refine",
  "participantId": "192571eb-71f5-4e69-9808-b1dedb71bdf6",
  "trialId": "f8523735-82a2-4926-9e7a-26a78eb13a19",
  "summary": {
    "successfulRuns": 3,
    "supportOutputIds": [101, 102, 103],
    "modelName": "gemini-2.0-flash",
    "allUsedFallback": false,
    "allProviderAttempted": true,
    "triggers": ["hesitation"],
    "observedComponentTypeSets": [
      ["Stack", "StepRail", "InspectPanel", "ConnectorLine"],
      ["Stack", "StepRail", "InspectPanel"],
      ["Stack", "StepRail", "InspectPanel", "ConnectorLine", "AnchoredTooltip"]
    ]
  },
  "selectedExample": {
    "requestId": "507d8f6e-525b-4478-8870-e9ac1be8e625",
    "supportOutputId": 103,
    "supportOutputCreatedAt": "2026-04-04T21:21:21.820Z",
    "trigger": "hesitation",
    "generation": {
      "modelName": "gemini-2.0-flash",
      "usedFallback": false,
      "fallbackReason": null,
      "providerAttempted": true,
      "catalogVersion": "catalog-v3",
      "componentTypes": [
        "Stack",
        "StepRail",
        "InspectPanel",
        "ConnectorLine",
        "AnchoredTooltip"
      ]
    }
  },
  "inputExcerpt": {
    "participantSnapshot": {
      "scenarioId": "slides-outline-refine",
      "slideOrder": [
        "slide-b-meeting",
        "slide-b-policy",
        "slide-b-proof",
        "slide-b-vote"
      ]
    }
  },
  "persistedOutput": {
    "catalogVersion": "catalog-v3",
    "spec": {
      "version": 1,
      "root": {
        "type": "Stack",
        "props": {},
        "children": [
          {
            "type": "StepRail",
            "props": {
              "targetIds": [
                "slide-b-meeting",
                "slide-b-policy",
                "slide-b-proof",
                "slide-b-vote"
              ]
            }
          }
        ]
      }
    }
  },
  {
    "type": "InspectPanel",
    "props": {
      "targetId": "deck-context-bar",
      "title": "Suggested Flow",
      "summary": "Try this order: meeting context -> relevant policy -> proof the society  

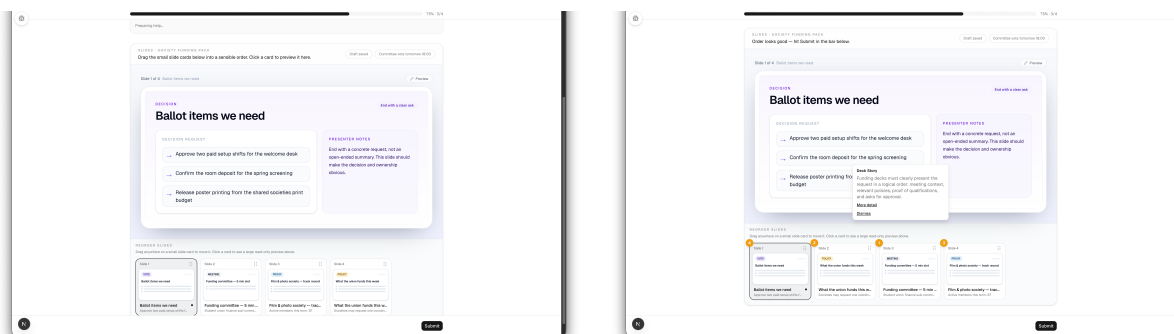
        ↳ qualifies -> ballot-ready asks. Frame the vote first, then explain the rules, show why  

        ↳ this society qualifies, and end with clear requests.",
      "details": [
        "Opening with context sets the stage.",
        "Highlighting policy grounds the story in the union's rules.",
        "Evidence builds confidence in the society's track record.",
        "Ending with concrete requests ensures a focused discussion."
      ]
    }
  }
}
```



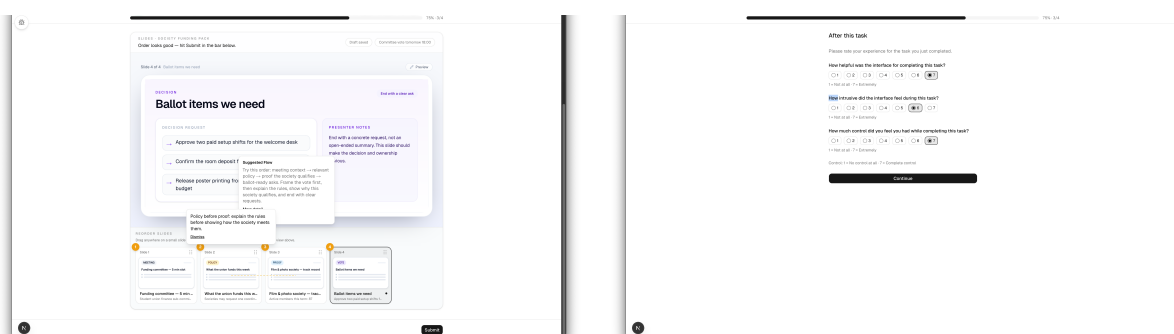
(a) Baseline post-trial questionnaire immediately after task completion. (b) Start of the ephemeral-condition trial, which warns that temporary help may appear.

Figure 11: Participant-flow walkthrough, part 2: transition from the baseline trial to the ephemeral trial.



(a) Ephemeral-condition task before the support overlay expands. (b) Generated support visible as an anchored explanation and ordered markers over the slide strip.

Figure 12: Participant-flow walkthrough, part 3: ephemeral assistance becoming visible during the task.



(a) Ephemeral overlay aligned to the current slide and reorder strip. (b) Expanded support details combining an inspect panel, tooltip, and ordering cues.

Figure 13: Participant-flow walkthrough, part 4: richer support states within the ephemeral trial.

[illegible]

10. Research the four main interfaces that implement the `Runnable` interface. Which interface would you expect to find on `Runnable` class in Java SE 8?

☐ `Runnable`

☐ `Runnable2`

☒ `RunnableAndCallable`

☐ `RunnableAndFuture`

11. Research the four main interfaces that implement the `Callable` interface. Which interface would you expect to find on `Callable` class in Java SE 8?

☐ `Callable`

☐ `Callable2`

☒ `CallableAndFuture`

☐ `CallableAndRunnable`

12. Research the four main interfaces that implement the `Future` interface. Which interface would you expect to find on `Future` class in Java SE 8?

☐ `Future`

☐ `Future2`

☒ `FutureAndCallable`

☐ `FutureAndRunnable`

13. Research the four main interfaces that implement the `FutureTask` interface. Which interface would you expect to find on `FutureTask` class in Java SE 8?

☐ `FutureTask`

☐ `FutureTask2`

☒ `FutureTaskAndCallable`

☐ `FutureTaskAndRunnable`

14. Research the four main interfaces that implement the `Executor` interface. Which interface would you expect to find on `Executor` class in Java SE 8?

☐ `Executor`

☐ `Executor2`

☒ `ExecutorAndFuture`

☐ `ExecutorAndRunnable`

15. Research the four main interfaces that implement the `ExecutorService` interface. Which interface would you expect to find on `ExecutorService` class in Java SE 8?

☐ `ExecutorService`

☐ `ExecutorService2`

☒ `ExecutorServiceAndFuture`

☐ `ExecutorServiceAndRunnable`

16. Research the four main interfaces that implement the `ThreadPoolExecutor` interface. Which interface would you expect to find on `ThreadPoolExecutor` class in Java SE 8?

☐ `ThreadPoolExecutor`

☐ `ThreadPoolExecutor2`

☒ `ThreadPoolExecutorAndFuture`

☐ `ThreadPoolExecutorAndRunnable`

17. Research the four main interfaces that implement the `ThreadLocalRandom` interface. Which interface would you expect to find on `ThreadLocalRandom` class in Java SE 8?

☐ `ThreadLocalRandom`

☐ `ThreadLocalRandom2`

☒ `ThreadLocalRandomAndFuture`

☐ `ThreadLocalRandomAndRunnable`

18. Research the four main interfaces that implement the `ConcurrentHashMap` interface. Which interface would you expect to find on `ConcurrentHashMap` class in Java SE 8?

☐ `ConcurrentHashMap`

☐ `ConcurrentHashMap2`

☒ `ConcurrentHashMapAndFuture`

☐ `ConcurrentHashMapAndRunnable`

19. Research the four main interfaces that implement the `ConcurrentMap` interface. Which interface would you expect to find on `ConcurrentMap` class in Java SE 8?

☐ `ConcurrentMap`

☐ `ConcurrentMap2`

☒ `ConcurrentMapAndFuture`

☐ `ConcurrentMapAndRunnable`

20. Research the four main interfaces that implement the `ConcurrentMap` interface. Which interface would you expect to find on `ConcurrentMap` class in Java SE 8?

☐ `ConcurrentMap`

☐ `ConcurrentMap2`

☒ `ConcurrentMapAndFuture`

☐ `ConcurrentMapAndRunnable`

(a) Beginning of the final comparative questionnaire after both trials are complete. (b) Completed final questionnaire with comparative judgments recorded.

Figure 14: Participant-flow walkthrough, part 5: final comparison questions at the end of the session.

A.5 Interpretive Note

These listings are sufficient to show the main route from raw study records to reported results: first the valid sample was defined consistently, then paired statistical tests were run, then publication-quality figures were generated, and finally the resulting numerical outputs were written into LaTeX commands consumed by the manuscript itself. This is the core evidence needed to demonstrate that the quantitative results chapter was generated through a reproducible analysis workflow rather than through manual transcription.